arXiv:submit/7611927 [cs.AI] 20 May 2026

BBOWP-Bench: Evaluating LLMs on Black-Box Optimization Word Problems

Yutaro Yamada* **Kei Hiroshima*** **Nozomu Yoshinari***
Kento Uchida **Shinichi Shirakawa**
Yokohama National University
{yamada-yutaro-dw,hiroshima-kei-st,yoshinari-nozomu-ry}@ynu.jp
{uchida-kento-fz,shirakawa-shinichi-bg}@ynu.ac.jp

Abstract

Formulating an optimization problem strongly affects the quality of the final solution, yet good formulations usually require substantial expertise. Recent studies have therefore examined how to automatically derive optimization problems from natural-language descriptions, but existing benchmarks focus on settings where objectives and constraints can be written explicitly as mathematical expressions. Many practically important problems, including hyperparameter optimization in machine learning, are naturally treated as black-box optimization (BBO) problems, in which only objective values are observable, and the functional form is unavailable. In BBO, the search space design, a part of the problem formulation, and the selection of the optimization algorithm are crucial for problem-solving. Automating these processes with large language models (LLMs) is a significant challenge. This paper introduces Black-Box Optimization Word Problems (BBOWP), a novel problem setting in which a system must infer both a search space and an optimization algorithm from a natural-language description of a black-box optimization task. To support research on this setting, we establish the BBOWP Benchmark Suite (BBOWP-Bench), a dataset and evaluation framework for BBOWP. Each instance combines a natural-language problem description, an executable evaluation environment, and a human-designed baseline formulation, allowing evaluation of both search-space design and algorithm selection. The benchmark covers 23 instances from four different application domains and supports reproducible assessment through actual black-box optimization runs. Using this benchmark, we provide the first evaluation of LLMs and show that current LLMs are capable of selecting suitable algorithms based on the given evaluation budget. However, they sometimes struggle with search space design, particularly in identifying important variables and balancing their ranges when the problem description is less informative or the search space is highly problem-specific. Our code and dataset are available at `https://github.com/shiralab/bbowp-bench`.

1 Introduction

Formulating an optimization problem has a major impact on solution quality. However, identifying the key elements of a problem, such as design variables and objectives, and then expressing them in a solver-ready form usually requires substantial expertise. Prior work has therefore studied the automatic formulation of optimization problems from natural-language descriptions. Recent datasets and benchmarks cover a range of optimization classes, from linear programming settings such as NL4Opt [Ramamonjison et al., 2022b] to more recent resources for nonlinear optimiza-

*Equal contribution.

Preprint.

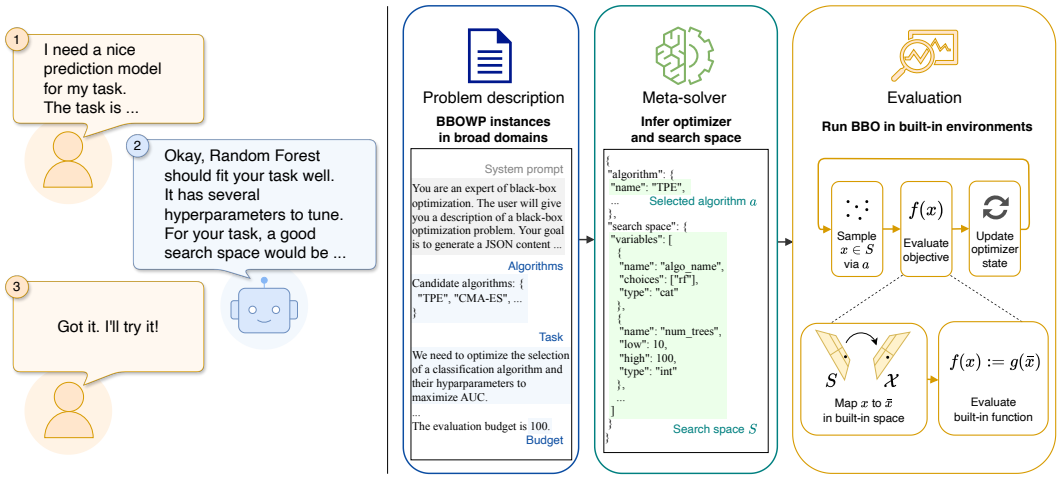


Figure 1: **(Left)** A typical scenario considered in BBOWP. Users ask a meta-solver to formulate a BBO problem for their task, but they may not have an implemented objective function when they describe the task. **(Right)** Overview of the BBOWP Benchmark Suite. Given a problem description d that includes information such as the maximum number of function evaluations $N_{\max}$, a meta-solver determines an appropriate optimization algorithm a and search space S. Then, the formulation is evaluated through actual optimization runs, using a built-in objective function via identifier mapping.

tion [Huang et al., 2025b, Yang et al., 2024, Huang et al., 2025a, Lu et al., 2025], and support the evaluation on tasks such as extracting variables, objectives, and constraints from text.

This line of work has produced useful datasets and evaluation settings, but existing benchmarks still target problem classes in which objectives and constraints can be written explicitly as mathematical expressions. In contrast, many important real-world tasks, including hyperparameter optimization, molecular design, and structural shape optimization, are more naturally cast as black-box optimization (BBO) problems. In BBO, only objective values are observable, and the underlying functional form is unknown or intractable. As a result, progress in auto-formulation for BBO requires not only new methods but also benchmark datasets that make formulation quality and downstream optimization performance measurable.

In parallel, it has been argued that there is a need for metadata-rich BBO benchmarks [Song et al., 2024]. Existing BBO benchmarks, such as COCO [Hansen et al., 2021], contain limited metadata describing the problem context. However, in practice, utilizing prior knowledge is essential to realize efficient optimization, and leveraging large language models (LLMs) to exploit such information can be a key direction. This viewpoint also motivates the development of a BBO benchmark with task information written in natural language.

Building on these research streams, we introduce Black-Box Optimization Word Problems (BBOWP), a novel problem setting for auto-formulation in BBO domains. In BBOWP, a system receives a natural-language problem description and must infer both a search space and a suitable optimization algorithm. To accelerate research on BBOWP, we develop the BBOWP Benchmark Suite (BBOWP-Bench), which pairs natural-language problem descriptions with design-variable domain definitions, executable evaluation environments, and human-designed reference formulations. We also design an evaluation protocol that supports both comparison against a human baseline and assessment through actual BBO runs. Using this framework, we conduct the first benchmark study of several LLMs and find that they can select suitable algorithms for the evaluation budget when provided with detailed problem descriptions. However, they often struggle to identify important variables and set appropriate ranges when problem descriptions are less informative.

Our contributions are summarized as follows: (1) We propose BBOWP, a new problem setting in which a system must infer both a search space and a suitable optimization algorithm for a BBO task from its natural-language description. (2) We introduce the BBOWP-Bench, a dataset for BBOWP that pairs natural-language problem descriptions with executable evaluation environments and human-designed baseline formulations. (3) We design an evaluation protocol that supports both

formulation-level comparison and downstream assessment through actual BBO runs. (4) We provide the first benchmark study of several LLMs on BBOWP and show that several task settings remain challenging for current LLMs.

2 Related Work

Constructing solver-ready formulations from natural-language problem descriptions requires substantial expertise. To reduce this reliance on experts, recent work has developed datasets and benchmarks for auto-formulation of optimization problems [Jiang et al., 2025, Ramamonjison et al., 2022a,b, Xiao et al., 2024, Ahmaditeshnizi et al., 2024, Huang et al., 2025b, Yang et al., 2024, Huang et al., 2025a, Lu et al., 2025]. Most of these studies focus on linear programming, integer programming, and mixed-integer linear programming [Ramamonjison et al., 2022a,b, Huang et al., 2025b], with more recent extensions to nonlinear programming [Huang et al., 2025a, Yang et al., 2024, Lu et al., 2025]. Despite this progress, many real-world optimization problems do not admit explicit symbolic objectives. Hyperparameter optimization, expensive simulations, materials discovery, and human-in-the-loop design are all naturally modeled as BBO problems. Our work extends the auto-formulation perspective to this broader setting.

Closely related but distinct text-to-optimization studies include [Li et al., 2025] and [Imajuku et al., 2026]. Li et al. [2025] studied automated code generation for a part of the BBO formulation from natural-language description, where the code computes objectives and constraints from simulation outputs. Imajuku et al. [2026] introduced ALE-Bench, which evaluates whether agents can improve performance on hard optimization tasks from task descriptions. In contrast, BBOWP focuses on an earlier formulation stage: given only a natural-language description, the system must infer not only a suitable search algorithm but also what search space should be exposed to the optimizer. Our work is therefore complementary to these studies as part of an end-to-end automated pipeline for BBO.

Algorithm selection is a central issue in BBO. Because the suitable optimizer depends on the search space, objective function landscape, and evaluation budget, the optimizer should be carefully chosen to match the task formulation rather than fixed in advance [Munoz et al., 2015, Kerschke et al., 2019]. BBOWP is closely related to this issue because solving a BBOWP instance requires inferring a search space and selecting an optimizer that is compatible with it.

3 Black-Box Optimization Word Problem (BBOWP)

BBOWP is formulated for settings in which the problem description may be written by a non-expert, so the underlying objective function is not assumed to be fully specified beforehand. Hence, the users may prepare their objective function *after* receiving an answer to BBOWP as shown in Figure 1 (left). Because any maximization problem can be converted into a minimization problem by minimizing the negative of the objective, we consider only minimization in what follows. In this paper, we assume BBO problems with a single and noiseless objective.

In BBOWP, the input is a natural-language problem description d. The description includes information such as the problem background, the target performance metric, the maximum number of function evaluations $N_{\max}$, and the set of available algorithms $\mathcal{A}$. Because the instruction may be written at a high level, the design variables are not always stated explicitly in d.

The output for BBOWP is a search space $S \in \mathcal{S}$ and an optimization algorithm $a \in \mathcal{A}$. Here, $\mathcal{S}$ denotes the class of admissible search spaces, and S denotes a formulation of the search space suitable for the target BBO problem described in d. For instance, the system for solving BBBOWP determines the number, names, types, and ranges of design variables as the search space design, aligning with the problem description. We include algorithm selection in the output because the suitability of an optimizer depends on the problem formulation and the available evaluation budget.

The quality of the answer (a, s) can be measured by the final solution quality when running the algorithm a to minimize the objective function on the search space s within the given budget $N_{\max}$. Namely, BBOWP asks for an optimizer and a search space that minimizes the expected objective value attained within the given budget: $(a^*, S^*) = \mathrm{argmin}_{a \in \mathcal{A},\, S \in \mathcal{S}}\ \mathbb{E}\left[\mathcal{V}(a, S, N_{\max})\right]$, where $\mathcal{V}(a, S, N_{\max})$ denotes the best objective value obtained by running algorithm a under the search space S with budget $N_{\max}$, evaluated on the underlying performance metric intended by d.

Because black-box optimizers are typically stochastic, $\mathcal{V}$ is generally a random variable. We study the performance of policies that output a suitable optimization algorithm a and a search space S for a given input d. We refer to such a policy as a meta-solver.

4 BBOWP Benchmark Suite

The input of BBOWP is a problem description written in natural language. Existing BBO benchmark datasets such as COCO [Hansen et al., 2021] and NeuroEvoBench [Lange et al., 2023] do not include natural-language descriptions of the problems, which makes them unsuitable for evaluating performance on BBOWP. To address this limitation, we propose the BBOWP-Bench, a benchmark dataset designed specifically for the BBOWP task. An overview of the framework is shown in Figure 1 (right).

4.1 Benchmark Design

At the abstract level, BBOWP does not assume that the search space is fixed in advance. Therefore, a meta-solver is free to design the search space, deciding which variables to optimize and how to identify them, based on the natural-language description and its intended optimization objective. Ideally, the evaluation should be able to assess any search space generated by a meta-solver. In principle, this could be done by having a human interpret the meta-solver's output and implement the corresponding objective interface for that formulation. However, such a procedure is impractical for a benchmark that must rely on pre-built execution environments and support automatic evaluation. The central design choice of the BBOWP-Bench is therefore to approximate this ideal evaluation protocol using a fixed executable objective together with an adapter that maps generated search spaces to the variable space expected by the executable environment.

The benchmark provides pre-built executable objectives and their problem descriptions derived from existing real-world BBO benchmarks (see Section 4.2). For a given description d, the objective $g_d : \mathcal{X}_d \to \mathbb{R}$ is defined on a sufficiently large canonical design space $\mathcal{X}_d = \mathcal{X}_{l_1} \times \cdots \times \mathcal{X}_{l_I}$. Each component is indexed by an identifier l_i representing the semantic role of a design variable. For example, one such identifier may be `learning_rate`. The benchmark also specifies default values $x_{l_i}^{\mathrm{def}} \in \mathcal{X}_{l_i}$.

By contrast, a meta-solver outputs a search space $S = S_{l'_1} \times \cdots \times S_{l'_J}$ together with an optimization algorithm. Because $\mathcal{X}_d$ is not always given to the meta-solver directly, the generated identifiers l'_j need not coincide with the canonical identifiers l_i. The meta-solver may omit relevant variables, introduce irrelevant ones, or use alternative names for the canonical executable names. Each generated variable additionally specifies its type, domain, and choice of sampling scale, either linear or logarithmic. If a logarithmic scale is selected, the optimization algorithm searches in the transformed space, and the realized value is mapped back before evaluation. The domain is defined by the upper and lower bounds of the search range for a design variable or by a singleton set for a constant value.

To evaluate a point $x = (x_{l'_1}, \ldots, x_{l'_J}) \in S$, the benchmark resolves correspondence between generated and canonical variables through an equivalence relation $\sim$ over identifiers. For each canonical variable l_i, let $M_i = \{j \in \{1, \ldots, J\} \mid l_i \sim l'_j\}$ denote the set of generated variables matched to l_i. When this correspondence is not ambiguous, the benchmark constructs a canonical point $\bar{x} = (\bar{x}_{l_1}, \ldots, \bar{x}_{l_I}) \in \mathcal{X}_d$ by

$$\bar{x}_{l_i} = \begin{cases} x_{l'_j} & \text{if } M_i = \{j\} \\ x_{l_i}^{\mathrm{def}} & \text{if } M_i = \emptyset \end{cases} , \quad . \tag{1}$$

The benchmark then evaluates the generated point through the induced wrapper objective

$$f_d(x) = \begin{cases} g_d(\bar{x}) & \text{if } |M_i| \le 1 \text{ for all } i, \text{ and if } x \text{ can be cast to } \bar{x} \in \mathcal{X}_d \\ \omega_d & \text{otherwise} \end{cases} , \tag{2}$$

where ω_d denotes the worst objective value for task d (∞ for minimization and $-\infty$ for maximization). Here, omitted variables are filled with task-specific default values, and irrelevant generated variables are ignored unless they create an ambiguity in the correspondence. We note that the worst objective value is returned when the canonical point $\bar{x}$ lies outside the canonical design space $\mathcal{X}_d$, i.e., $\bar{x} \notin \mathcal{X}_d$.

This construction is the key design choice in the BBOWP-Bench. It enables evaluation of a broad range of generated search spaces using a fixed executable objective g_d. In this sense, the benchmark approximates the ideal flexible objective needed by the abstract task definition without requiring a new execution environment for every generated formulation. Further details of the benchmark design are discussed in Appendix C.

BBOWP-Bench also includes the list of candidate black-box optimizers. The system for solving BBOWP is expected to select a suitable one from the candidate list in addition to designing the search space. Then, the output, a pair of a selected algorithm and a designed search space (a, S), can be automatically evaluated by running the algorithm a on the wrapper objective (2) with S.

4.2 Data Description

Table 1: Examples of the instances in BBOWP-Bench.

Source	Task	Scenario	Dim	Float	Int	Cat
YAHPO	3	HPO of Ranger	8	✓	✓	✓
	4	HPO of SVM	6	✓	✓	✓
Olympus	14	S_NAr reaction	4	✓		
	15	HPLC system	6	✓		
MECHBench	17	Star-shaped crash box	5	✓		
MuJoCo	19	Inverted Pendulum	5	✓		
	20	Inverted Double Pendulum	10	✓		

We created BBOWP instances from four sources: YAHPO Gym [Pfisterer et al., 2022], Olympus [Häse et al., 2021, Hickman et al., 2023], MECHBench [Rodríguez et al., 2025], and MuJoCo-based continuous-control tasks [Todorov et al., 2012]. YAHPO Gym is a benchmark dataset for hyperparameter optimization (HPO) of machine learning models. Olympus covers experimental science domains, such as materials science and drug discovery, and consists of tasks that emulate real-world physical experiments and chemical simulations. MECHBench is a benchmark dataset for shape optimization in structural mechanics and consists of tasks that involve designing structures modeled after vehicle components to achieve high crash-safety performance. MuJoCo provides physics-based continuous-control tasks in simulated robotic environments.

From these sources, we built 7 tasks from YAHPO Gym, 9 from Olympus, 2 from MECHBench, and 5 from MuJoCo, yielding 23 tasks in total. We carefully extract the appropriate tasks for BBOWP-Bench by excluding those with ambiguous variable names (HPO of the glmnet scenario in YAHPO) and those with implementation problems inconsistent with the paper (Alkoxylation scenario in Olympus). Examples of scenarios and design-variable types are shown in Table 1 (see Table 4 for complete details). The execution environment for each benchmark is prepared as a separate Docker image for each problem domain.

For each problem instance, we prepare a JSON record containing the canonical variables, their domains, and default values used by the executable environment. We also prepare three problem descriptions with different difficulty levels: *Easy*, *Intermediate*, and *Hard*. The *Easy* description explicitly includes an explanation of the full canonical design space, the *Intermediate* description includes partial clues from which the canonical design space can be inferred, and the *Hard* description contains no direct information about the canonical design space and instead describes only the task background and optimization objective.

To define the equivalence relation $\sim$ among identifiers, we also create a list of semantically equivalent names for each design variable. For example, for a design variable `learning_rate`, we include synonymous identifiers such as `lr` and `step_size`.

Each task is accompanied by a search space and an optimization algorithm selected and verified by multiple BBO experts as the human baseline. It also provides the reasons behind the choice of variables, ranges, scales, and optimizer under different settings for evaluation budget: *Small* with $N_{\max} = 100$, *Large* with $N_{\max} = 10{,}000$, where $N_{\max}$ is the maximum number of function evaluations. The experts designed the search space to avoid missing important variables and to balance its size with the given budget, and selected the optimization algorithm based on variable types and budget. Note that these human baselines were created by referring to the *Easy* description. This information makes the benchmark useful as a reference baseline, a source of few-shot examples, and a basis for further analysis. We emphasize that the expert formulation is not intended to represent the optimal answer. Rather, it serves as a high-quality reference baseline for validation and analysis. See Appendix D for details of the input and output of BBOWP-Bench.

Table 2: Comparison of optimization performance in *Easy* and *Hard* difficulties with *Small* budget. We present the results for several tasks here; the complete results are shown in Section G.1.

		YAHPO		Olympus		MECHBench
Difficulty	Model	Task 3 (↑)	Task 4 (↑)	Task 14 (↓)	Task 15 (↑)	Task 17 (↓)
Easy	Gemini 3 Flash	0.92	**0.98**	**0.19**	**2695.47**	**-11687.66**
	Gemini 3.1 Pro	0.87	0.94	0.20	2364.52	**-11687.66**
	GPT-5 mini	0.94	0.97	0.32	2310.73	-10758.91
	GPT-5.2	**0.95**	0.94	0.20	2246.71	-10758.91
Hard	Gemini 3 Flash	**0.88**	**0.95**	0.89	$-\infty$	**-8535.27**
	Gemini 3.1 Pro	0.82	**0.95**	2.27	$-\infty$	-8528.84
	GPT-5 mini	0.84	0.93	0.79	$-\infty$	∞
	GPT-5.2	0.81	**0.95**	**0.78**	**1320.79**	-8507.17
Human Baseline		0.91	0.98	0.20	2366.82	-11740.92

5 Experiments

We assess the performance of representative LLMs as meta-solvers and demonstrate the usefulness of BBOWP-Bench.

5.1 Experimental Setup

Evaluated Meta-Solvers. To investigate the impact of model size and reasoning capability on BBOWP, we evaluated four representative LLMs: Gemini 3 Flash [Google DeepMind, 2025], Gemini 3.1 Pro [Google DeepMind, 2026], GPT-5 mini [OpenAI, 2025b], and GPT-5.2 [OpenAI, 2025a].

Prompting and Difficulty Levels. For each problem, the LLM receives a prompt containing an overview of the BBOWP task, the required JSON output format, the problem description, and the target maximum number of function evaluations (budget). To rigorously assess the models' capabilities to build a search space, we evaluated the LLMs across three difficulty levels of problem descriptions: *Easy*, *Intermediate*, and *Hard* (see Section 4.2). We evaluated them in both *zero-shot* and *one-shot* settings and discuss the results of the *zero-shot* setting in this section. The results of *one-shot* are provided in Section G.2. Complete prompt templates are provided in Section D.1.

Candidate Algorithms and Budget Conditions. The LLMs are instructed to select an optimization algorithm from a predefined pool of standard optimizers for BBO: CMA-ES [Hansen and Ostermeier, 1996], CMA-ES with Margin (CMA-ESwM) [Hamano et al., 2022], CatCMA [Hamano et al., 2024], CatCMA with Margin (CatCMAwM) [Hamano et al., 2025], Differential Evolution (DE) [Storn and Price, 1997], Nelder-Mead [Nelder and Mead, 1965], Particle Swarm Optimization (PSO) [Kennedy and Eberhart, 1995], Bayesian optimization with Gaussian process regression (GP) [Jones et al., 1998], SMAC3 [Lindauer et al., 2022], and Tree-structured Parzen Estimator (TPE) [Bergstra et al., 2011]. We query the LLMs to design search spaces and select algorithms for three distinct budgets.

Execution of Optimization. For the end-to-end evaluation through actual optimization runs, we use the predefined script to execute the optimization according to the JSON specification generated by a LLM. The optimization script is implemented using OptunaHub library [Ozaki et al., 2026]. While the optimization runs on the generated search space, actual function evaluations are performed using the built-in function as described in Section 4.1. We ran the optimization algorithm for each search space once and collected the best evaluation values obtained during the optimization. For *Small* budget settings, the optimization runs are executed across all 23 tasks. For *Large* budget settings, the optimization runs are executed across 21 tasks other than the MECHBench tasks (Tasks 17 and 18) since each evaluation takes a minute or more in those tasks.

5.2 Behavioral Analysis of LLMs as Meta-Solvers

Sensitivity to Problem Domains and Difficulties. A critical requirement for a meta-solver is the ability to infer the variables to be optimized when problem descriptions are ambiguous. Table 2

presents the optimization performance in *Easy* and *Hard* difficulties with *Small* budget. In the *Easy* setting, where the full canonical design spaces are explicitly provided in the prompt, most models act as highly capable instruction followers, successfully mapping the text to valid search spaces across all domains. However, in the *Hard* setting, where models must infer the design variables and bounds entirely from an ambiguous background description, a significant gap across application domains emerges. For Tasks 3 and 4 in HPO (YAHPO Gym), we observed that LLMs successfully reconstruct valid search spaces even in the *Hard* setting. For example, for Task 4 (HPO of SVM), all LLMs include the key variables `cost`, `kernel`, and `gamma` with appropriate ranges (see Table 17 for details). Conversely, for tasks in Olympus and MECHBench, models frequently design invalid or ineffective search spaces in the *Hard* setting. For example, Task 14 in Olympus includes four variables, `residence_time`, `ratio`, `concentration`, and `temperature` (see Table 18). This task treats a chemical experiment under pressurization and thus requires to keep `temperature` high to accelerate the reaction. Against the canonical domain $[60.0, 140.0]$ for `temperature`, Gemini 3.1 Pro designs $[20.0, 140.0]$ while Gemini 3 Flash sets the range to $[0.0, 250.0]$. Given that the worst value is returned when a solution lies outside the canonical domain, Gemini 3.1 Pro successfully designs its search space suitable for the task scenario, and thus achieves a better score than other models in this task. Appendix H provides the details of the actual designed search spaces by LLMs. We note that several search spaces designed by LLMs from *Hard* and *Intermediate* descriptions contain design variables that lack a mapping to their canonical executable variable names. Our framework incorporates alias dictionaries that map each design variable in an LLM-generated search space to its canonical counterpart by covering as broad a range of name variations as possible. Developing an effective method to map variables more precisely is left for future work.

Budget Awareness in Algorithm Selection. A desired capability in BBOWP is the ability to select an algorithm suited to the computational budget. We analyze the distribution of optimization algorithms selected by the LLMs with *Small* ($N_{\max} = 100$) versus *Large* ($N_{\max} = 10{,}000$) evaluation budgets. The detailed figure of comparison can be found in Appendix F. We observe a highly rational shift in model behavior against the evaluation budget, confirming that all four LLMs possess a strong degree of budget awareness. For example, the proportion of selecting Bayesian optimization algorithms (GP, SMAC3, and TPE) by GPT-5.2 is 56.5% for *Small* budget, whereas it decreases to 30.4% for *Large* budget. More clearly, Gemini 3.1 Pro selects these algorithms at a rate of 78.3% for *Small* budget, whereas 30.4% for *Large* budget. This behavior indicates that LLMs correctly recognize that with an abundant evaluation budget, population-based evolutionary algorithms are preferable for exploiting their scalability and convergence ability.

5.3 Scale-Invariant Evaluation of Search Space and Selected Algorithm

Evaluating meta-solver performance across vastly different application domains presents a significant aggregation challenge, as raw objective values have incommensurable scales and bounds. To evaluate the quality of the meta-solvers' formulations composed of the search space S and the selected algorithm a in a scale-invariant manner, we propose a quantile-based evaluation metric.

Formulation Ranking Score (FRS). We measure the probability that the best objective function value found based on the meta-solver's answer is superior to the results obtained from search spaces and optimization algorithms obeying a certain distribution. Formally, let $y(a, S)$ denote the best objective function value achieved by running an optimization algorithm $a \in \mathcal{A}$ on a search space $S \in \mathcal{S}$ for a given task. Assuming a minimization problem without loss of generality, we define *Formulation Ranking Score (FRS)* for a specific configuration (a, S) as follows:

$$\mathrm{FRS}(a, S) = \Pr_{(a', S') \sim P_{(a,S)}} \left(y(a, S) \leq y(a', S') \right) \quad , \tag{3}$$

where $P_{(a,S)}$ indicates the reference distributions of algorithms and search spaces. This metric quantifies the quality of the configuration (a, S) relative to those distributed according to $P_{(a,S)}$.

In practice, since this exact probability cannot be computed analytically, we employ a Monte Carlo approximation to Equation (3) to estimate it:

$$\widehat{\mathrm{FRS}}(a, S) = \frac{1}{|\mathcal{R}|} \sum_{(a', S') \in \mathcal{R}} \mathbb{I}\left\{ y(a, S) \leq y(a', S') \right\} \quad , \tag{4}$$

Table 3: Comparison of FRS between different models in the *Small* budget setting. Each value is the average of FRS across Tasks 3, 14, and 20. The average FRS of the human baseline is **0.964**. Full results are provided in Appendix I. **Bold** and Underline indicate the best and the second best score among the four models, respectively.

Difficulty	Gemini 3 Flash	Gemini 3.1 Pro	GPT-5 mini	GPT-5.2
Easy	**0.965**	0.936	0.855	0.843
Intermediate	0.434	0.557	**0.730**	0.416
Hard	**0.728**	0.133	0.588	0.503
Avg.	0.709	0.542	**0.724**	0.588

where $\mathbb{I}(\cdot)$ is the indicator function, and $\mathcal{R}$ represents a finite reference set of optimization algorithms and search spaces drawn from $P_{(a,S)}$. In our experiments, to construct the reference set $\mathcal{R}$, we sampled the search space S' by sampling variable bounds and categorical choices from the canonical decision space according to a uniform distribution, while the optimization algorithm a' was randomly sampled from the candidate algorithms defined in Section 5.1, excluding high-overhead solvers: SMAC3 and Bayesian optimization with Gaussian process. The number of samples is $|\mathcal{R}| = 100,000$. The approximated FRS indicates the proportion of elements in the reference set $\mathcal{R}$ that perform worse than the given configuration (a, S). As the range of FRS is $[0, 1]$, we can measure the performance of meta-solvers on tasks with different domains in the same scale, regardless of the scales or landscape complexities of the underlying objective functions. For simplicity, we hereafter denote $\widehat{\text{FRS}}$ as FRS.

FRS-based Comparison. Using the FRS metric, we evaluate the performance of the LLMs across the specified tasks under the *Small* budget ($N_{\text{max}} = 100$). The aggregated results in Table 3 reveal significant variance in meta-solver performance depending on problem difficulty. Most notably, GPT-5 mini achieves the highest overall average FRS (0.724), closely followed by Gemini 3 Flash (0.709). Both significantly outperform the larger foundation models, with Gemini 3.1 Pro demonstrating the lowest overall score (0.542).

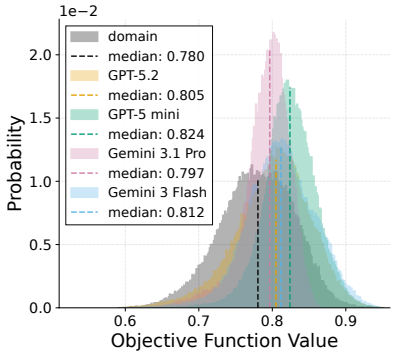
Analyzing performance across difficulty levels uncovers distinct model behaviors. In the *Easy* setting, where full canonical bounds are explicitly provided in the prompt, Gemini 3 Flash (0.965) and Gemini 3.1 Pro (0.936) achieve the highest scores, leveraging capabilities to map the given description into highly optimized search spaces. For instance, Gemini 3.1 Pro successfully prunes its search space by fixing some variables to constant values, as the human baseline also does (see Table 21 for details).

However, the *Hard* setting sometimes exposes severe fragility in large models. While Gemini 3 Flash achieves an FRS of 0.728, Gemini 3.1 Pro achieves 0.133. This is because it performs poorly on Tasks 14 and 20. For example, in Task 14, Gemini 3.1 Pro shows the worst performance among the four LLMs since it fails to specify the variable `concentration`, which the other three models include in their search spaces (see Table 18 for details).

5.4 Isolating Search Space Quality

While the FRS metric evaluates the quality of a pair consisting of a selected algorithm a and a search space S, it does not disentangle whether high performance stems from a robust optimization algorithm or a well-designed search space. In this section, we therefore isolate the quality of the generated search space S independently of the optimization algorithm. We achieve this by analyzing the empirical probability density function (EPDF) of objective function values within the search space designed by the LLMs. For the same three representative tasks analyzed in Section 5.3, we uniformly sample 100,000 points from both the canonical domain and each LLM-designed search space, evaluating the objective function value for every point.

Aggressive Dimensionality Reduction by Advanced LLMs. The left panel of Figure 2 illustrates the EPDFs of the objective function values for Task 3, HPO of Ranger in YAHPO. The gray curve represents the canonical domain, which exhibits a wide variance and a median objective value of 0.780. Because the canonical domain is broad, an optimization algorithm operating under a strict budget of $N_{\max} = 100$ is highly likely to waste evaluations in low-performing regions.



Variable	Domain	GPT-5 mini	Gemini 3.1 Pro
sample.fraction	[0.1, 1.0]	[0.2, 1.0]	[0.5, 1.0]
mtry.power	[1, 2000]	[50, 1000]	500
num.random splits	[1, 100]	[1, 100]	1
min.node.size	[1, 100]	[1, 100]	[1, 20]
splitrule	{gini, extratrees}	{gini, extratrees}	{gini, extratrees}
respect.unordered factors	{ignore, order, partition}	{ignore, order, partition}	ignore
imputation	{impute.mean, impute.median, impute.hist}	{impute.mean, impute.median, impute.hist}	impute.mean

Figure 2: Search space comparison designed by LLMs for Task 3 in *Easy* difficulty. **(Left)** Empirical probability density function (EPDFs) of objective function value over the canonical domain (gray) versus the search spaces designed by four LLMs. **(Right)** Comparison table of ranges or categorical choices for each variable in the canonical domain and the search spaces designed by GPT-5 mini (achieved the highest median) and Gemini 3.1 Pro (exhibits the worst median). Underline in the table indicates that the variable is set to a constant value. Gemini 3.1 Pro aggressively reduced the dimensionality by setting four variables to constant values.

The EPDFs over the search spaces designed by LLMs reveal the mechanical causes behind the FRS observed in Section 5.3. GPT-5 mini (green curve), which achieved the highest FRS overall, shifts the bulk of the probability mass rightward to a median of 0.824. It successfully covers a desirable region with high-quality solutions. Conversely, Gemini 3.1 Pro (red curve), which exhibited the worst overall FRS among the four LLMs, yields a lower median of 0.797. Its EPDF exhibits exceptionally high kurtosis, with a sharp and narrow peak strictly concentrated around its median, resulting in highly homogeneous objective function values.

The right panel of Figure 2 shows the canonical domain alongside the search spaces designed by GPT-5 mini and Gemini 3.1 Pro. This explicitly explains the distributional differences: while GPT-5 mini pruned the bounds to exclude only low-performing regions, Gemini 3.1 Pro designed the narrow search space by fixing four variables to constant values. By inadvertently collapsing these dimensions, the model eliminated the region necessary for the optimizer to find high-quality solutions, thereby trapping the optimization algorithm in a suboptimal subspace. See Appendix J for more results.

6 Conclusion

This paper proposed the Black-Box Optimization Word Problem (BBOWP), a novel problem setting requiring LLM to infer both a search space and an optimization algorithm from the natural-language description of the target BBO task, and established the BBOWP-Bench as the first dataset and evaluation framework for the BBOWP. Each problem instance in BBOWP-Bench provides a natural-language problem description, an executable evaluation environment, and a human-designed baseline formulation, enabling the evaluation of both search-space design and algorithm selection. Using BBOWP-Bench, we evaluated several LLMs and uncovered their ability for BBOWP. We also introduced FRS, a quantitative and intuitive evaluation metric, to evaluate search configurations, whereas we analyzed EPDF of objective function value for evaluating the generated search space. The empirical evaluations demonstrate that while current LLMs are capable of selecting suitable algorithms, they sometimes struggle with search space design, particularly in identifying important variables and balancing their ranges.

While this work opens a new frontier in BBO research, BBOWP-Bench has several limitations that suggest future directions. A major limitation is the removal of generated variables whose identifiers are not included in the alias dictionaries. This may be resolved by using an additional LLM to map the generated identifier to a pre-defined identifier more appropriately. Moreover, BBOWP-Bench does not support providing conditional search space information to optimizers, which often appears in HPO tasks, indicating the need to improve our benchmark. In addition, because the optimization algorithm is selected from the set of algorithm candidates, expanding the set of algorithm candidates

or generating a new algorithm tailored to given tasks using an agent-based meta-solver is another possible future direction.

References

Ali Ahmaditeshnizi, Wenzhi Gao, and Madeleine Udell. OptiMUS: Scalable optimization modeling with (MI)LP solvers and large language models. In Ruslan Salakhutdinov, Zico Kolter, Katherine Heller, Adrian Weller, Nuria Oliver, Jonathan Scarlett, and Felix Berkenkamp, editors, *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 577–596. PMLR, 21–27 Jul 2024.

James Bergstra, Rémi Bardenet, Yoshua Bengio, and Balázs Kégl. Algorithms for hyper-parameter optimization. In *NeurIPS*, 2011.

Google DeepMind. Gemini 3 Flash Model Card. `https://deepmind.google/models/model-cards/gemini-3-flash/`, December 2025.

Google DeepMind. Gemini 3.1 Pro Model Card. `https://deepmind.google/models/model-cards/gemini-3-1-pro/`, February 2026.

Ryoki Hamano, Shota Saito, Masahiro Nomura, and Shinichi Shirakawa. CMA-ES with margin: lower-bounding marginal probability for mixed-integer black-box optimization. In *Proceedings of the Genetic and Evolutionary Computation Conference*, pages 639–647, 2022.

Ryoki Hamano, Shota Saito, Masahiro Nomura, Kento Uchida, and Shinichi Shirakawa. CatCMA: Stochastic optimization for mixed-category problems. In *Proceedings of the Genetic and Evolutionary Computation Conference*, GECCO '24, page 656–664, New York, NY, USA, 2024. Association for Computing Machinery. ISBN 9798400704949. doi: 10.1145/3638529.3654198. URL `https://doi.org/10.1145/3638529.3654198`.

Ryoki Hamano, Masahiro Nomura, Shota Saito, Kento Uchida, and Shinichi Shirakawa. CatCMA with margin: Stochastic optimization for continuous, integer, and categorical variables. In *Proceedings of the Genetic and Evolutionary Computation Conference*, pages 737–745, 2025.

N. Hansen, A. Auger, R. Ros, O. Mersmann, T. Tušar, and D. Brockhoff. COCO: A platform for comparing continuous optimizers in a black-box setting. *Optimization Methods and Software*, 36: 114–144, 2021. doi: https://doi.org/10.1080/10556788.2020.1808977.

Nikolaus Hansen and Andreas Ostermeier. Adapting arbitrary normal mutation distributions in evolution strategies: The covariance matrix adaptation. In *Proceedings of the IEEE International Conference on Evolutionary Computation*, pages 312–317. IEEE, 1996.

Florian Häse, Matteo Aldeghi, Riley J. Hickman, Loïc M. Roch, Melodie Christensen, Elena Liles, Jason E. Hein, and Alán Aspuru-Guzik. Olympus: a benchmarking framework for noisy optimization and experiment planning. *Machine Learning: Science and Technology*, 2(3):035021, July 2021. ISSN 2632-2153. doi: 10.1088/2632-2153/abedc8.

Riley Hickman, Priyansh Parakh, Austin Cheng, Qianxiang Ai, Joshua Schrier, Matteo Aldeghi, and Alán Aspuru-Guzik. Olympus, enhanced: benchmarking mixed-parameter and multi-objective optimization in chemistry and materials science, May 2023.

Chenyu Huang, Zhengyang Tang, Shixi Hu, Ruoqing Jiang, Xin Zheng, Dongdong Ge, Benyou Wang, and Zizhuo Wang. ORLM: A customizable framework in training large models for automated optimization modeling. *Operations Research*, 2025a.

Xuhan Huang, Qingning Shen, Yan Hu, Anningzhe Gao, and Benyou Wang. LLMs for mathematical modeling: Towards bridging the gap between natural and mathematical languages. In Luis Chiruzzo, Alan Ritter, and Lu Wang, editors, *Findings of the Association for Computational Linguistics: NAACL 2025*, pages 2678–2710, Albuquerque, New Mexico, April 2025b. Association for Computational Linguistics. ISBN 979-8-89176-195-7. doi: 10.18653/v1/2025.findings-naacl.146.

Yuki Imajuku, Kohki Horie, Yoichi Iwata, Kensho Aoki, Naohiro Takahashi, and Takuya Akiba. ALE-bench: A benchmark for long-horizon objective-driven algorithm engineering. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2026. URL https://openreview.net/forum?id=JCjGvbsOmQ.

Caigao Jiang, Xiang Shu, Hong Qian, Xingyu Lu, JUN ZHOU, Aimin Zhou, and Yang Yu. LLMOPT: Learning to define and solve general optimization problems from scratch. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=9OMvtboTJg.

Donald R Jones, Matthias Schonlau, and William J Welch. Efficient global optimization of expensive black-box functions. *Journal of Global Optimization*, 1998.

James Kennedy and Russell Eberhart. Particle swarm optimization. In *ICNN*, 1995.

Pascal Kerschke, Holger H Hoos, Frank Neumann, and Heike Trautmann. Automated algorithm selection: Survey and perspectives. *Evolutionary Computation*, 27(1):3–45, 2019.

Robert Tjarko Lange, Yujin Tang, and Yingtao Tian. NeuroEvoBench: Benchmarking evolutionary optimizers for deep learning applications. In *NeurIPS Datasets and Benchmarks Track*, 2023.

Yuchen Li, Handing Wang, Bing Xue, Mengjie Zhang, and Yaochu Jin. Solver-independent automated problem formulation via llms for high-cost simulation-driven design. *arXiv preprint arXiv:2512.18682*, 2025.

Marius Lindauer, Katharina Eggensperger, Matthias Feurer, André Biedenkapp, Difan Deng, Carolin Benjamins, Tim Ruhkopf, René Sass, and Frank Hutter. SMAC3: A versatile bayesian optimization package for hyperparameter optimization. *Journal of Machine Learning Research*, 2022. URL http://jmlr.org/papers/v23/21-0888.html.

Hongliang Lu, Zhonglin Xie, Yaoyu Wu, Can Ren, Yuxuan Chen, and Zaiwen Wen. OptMATH: A scalable bidirectional data synthesis framework for optimization modeling. In *Forty-second International Conference on Machine Learning*, 2025. URL https://openreview.net/forum?id=9P5e6iE4WK.

Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg S Corrado, and Jeff Dean. Distributed representations of words and phrases and their compositionality. *Advances in Neural Information Processing Systems*, 26, 2013.

Mario A. Munoz, Yuan Sun, Michael Kirley, and Saman K. Halgamuge. Algorithm selection for black-box continuous optimization problems: A survey on methods and challenges. *Information Sciences*, 317:224–245, 2015. doi: 10.1016/j.ins.2015.05.010.

J. A. Nelder and R. Mead. A simplex method for function minimization. *The Computer Journal*, 7 (4):308–313, 01 1965. ISSN 0010-4620. doi: 10.1093/comjnl/7.4.308. URL https://doi.org/10.1093/comjnl/7.4.308.

OpenAI. Update to GPT-5 System Card: GPT-5.2. https://deploymentsafety.openai.com/gpt-5-2, December 2025a.

OpenAI. GPT-5 System Card. https://cdn.openai.com/gpt-5-system-card.pdf, August 2025b.

Yoshihiko Ozaki, Shuhei Watanabe, and Toshihiko Yanase. OptunaHub: A platform for black-box optimization, 2026. URL https://arxiv.org/abs/2510.02798.

Florian Pfisterer, Lennart Schneider, Julia Moosbauer, Martin Binder, and Bernd Bischl. YAHPO Gym - an efficient multi-objective multi-fidelity benchmark for hyperparameter optimization. In Isabelle Guyon, Marius Lindauer, Mihaela van der Schaar, Frank Hutter, and Roman Garnett, editors, *Proceedings of the First International Conference on Automated Machine Learning*, volume 188 of *Proceedings of Machine Learning Research*, pages 3/1–39. PMLR, 25–27 Jul 2022. URL https://proceedings.mlr.press/v188/pfisterer22a.html.

Rindra Ramamonjison, Haley Li, Timothy Yu, Shiqi He, Vishnu Rengan, Amin Banitalebi-dehkordi, Zirui Zhou, and Yong Zhang. Augmenting operations research with auto-formulation of optimization models from problem descriptions. In Yunyao Li and Angeliki Lazaridou, editors, *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing: Industry Track*, pages 29–62, Abu Dhabi, UAE, December 2022a. Association for Computational Linguistics. doi: 10.18653/v1/2022.emnlp-industry.4. URL `https://aclanthology.org/2022.emnlp-industry.4/`.

Rindranirina Ramamonjison, Timothy Yu, Raymond Li, Haley Li, Giuseppe Carenini, Bissan Ghaddar, Shiqi He, Mahdi Mostajabdaveh, Amin Banitalebi-Dehkordi, Zirui Zhou, and Yong Zhang. NL4Opt competition: Formulating optimization problems based on their natural language descriptions. In Marco Ciccone, Gustavo Stolovitzky, and Jacob Albrecht, editors, *Proceedings of the NeurIPS 2022 Competitions Track*, volume 220 of *Proceedings of Machine Learning Research*, pages 189–203. PMLR, 28 Nov–09 Dec 2022b.

Iván Olarte Rodríguez, Maria Laura Santoni, Fabian Duddeck, Carola Doerr, Thomas Bäck, and Elena Raponi. Mechbench: A set of black-box optimization benchmarks originated from structural mechanics. *arXiv preprint arXiv:2511.10821*, 2025.

Xingyou Song, Yingtao Tian, Robert Tjarko Lange, Chansoo Lee, Yujin Tang, and Yutian Chen. Position: Leverage foundational models for black-box optimization. In Ruslan Salakhutdinov, Zico Kolter, Katherine Heller, Adrian Weller, Nuria Oliver, Jonathan Scarlett, and Felix Berkenkamp, editors, *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 46168–46180. PMLR, 21–27 Jul 2024.

Rainer Storn and Kenneth Price. Differential Evolution –a simple and efficient heuristic for global optimization over continuous spaces. *Journal of Global Optimization*, 1997.

Emanuel Todorov, Tom Erez, and Yuval Tassa. Mujoco: A physics engine for model-based control. In *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 5026–5033. IEEE, 2012. doi: 10.1109/IROS.2012.6386109.

Ziyang Xiao, Dongxiang Zhang, Yangjun Wu, Lilin Xu, Yuan Jessica Wang, Xiongwei Han, Xiaojin Fu, Tao Zhong, Jia Zeng, Mingli Song, and Gang Chen. Chain-of-experts: When LLMs meet complex operations research problems. In *The Twelfth International Conference on Learning Representations*, 2024.

Zhicheng Yang, Yiwei Wang, Yinya Huang, Zhijiang Guo, Wei Shi, Xiongwei Han, Liang Feng, Linqi Song, Xiaodan Liang, and Jing Tang. OptiBench meets resocratic: Measure and improve llms for optimization modeling. *arXiv preprint arXiv:2407.09887*, 2024.

A Details of the instances in BBOWP-Bench

Table 4 present the list of all instances in BBOWP-Bench.

Table 4: All instances in BBOWP-Bench. **Dim** indicates the number of canonical design variables. **Float**, **Int**, and **Cat** mean whether floating (continuous), integer, and categorical variables are included, respectively.

Source	Task	Scenario	Dim	Float	Int	Cat
YAHPO	1	HPO of MLP	7	✓	✓	
	2	HPO of RcppHNSW	6		✓	✓
	3	HPO of Ranger	8	✓	✓	✓
	4	HPO of RPART	5	✓	✓	✓
	5	HPO of SVM	6	✓	✓	✓
	6	HPO of XGBoost	14	✓	✓	✓
	7	Algorithm selection, HPO	36	✓	✓	✓
Olympus	8	N-benzylation	4	✓		
	9	Color mixing BOB	5	✓		
	10	Color mixing N9	3	✓		
	11	Fullerenes synthesis	3	✓		
	12	Photo PCE10 blend	4	✓		
	13	Photo WF3 blend	4	✓		
	14	S_NAr reaction	4	✓		
	15	HPLC system	6	✓		
	16	Suzuki reaction	4	✓		
MECHBench	17	Star-shaped crash box	5	✓		
	18	Three-point bending of layered beam	10	✓		
MuJoCo	19	Inverted Pendulum	5	✓		
	20	Inverted Double Pendulum	10	✓		
	21	Hopper	36	✓		
	22	Reacher	22	✓		
	23	Swimmer	18	✓		

B Code and Data Availability

BBOWP-Bench includes four existing assets: YAHPO Gym [Pfisterer et al., 2022], Olympus [Häse et al., 2021, Hickman et al., 2023], MECHBench [Rodríguez et al., 2025], and MuJoCo [Todorov et al., 2012]. YAHPO Gym and MuJoCo are licensed under Apache-2.0 License. Olympus and MECHBench are licensed under MIT License. Our codes are available under MIT License at `https://github.com/shiralab/bbowp-bench`. Our benchmark and dataset are also released under the Creative Commons Attribution 4.0 International License (CC BY 4.0) at `https://github.com/shiralab/bbowp-bench/benchmark`.

C Further Discussion on Benchmark Design

In building BBOWP-Bench, we made several decisions about the benchmark design. Here, we discuss other possibilities.

Evaluation Using Prebuilt Functions. Instead of using prebuilt functions, we had another option: building an objective function according to a given search space on the fly, using coding agents. This would make it possible to evaluate broader search spaces than those we considered. We did not adopt this approach due to two concerns. First, automatically generating objective functions is itself a challenging task and has been studied as a research topic in [Li et al., 2025]. This could confound the evaluation results, as it would entangle the capabilities of meta-solvers with those of coding agents. Second, coding agents are large, expensive, and frequently updated, which detracts from the portability and stability of the benchmark. We therefore prefer a more compact and stable design. For the same reason, we did not adopt LLM-as-a-Judge.

Identifier Matching Using Alias Lists. An obvious limitation of BBOWP-Bench is that valid design variables may be overlooked due to the limited coverage of the alias lists. Using vector representations of identifiers, such as Word2Vec [Mikolov et al., 2013], could alleviate this limitation. However, doing so would introduce a dependency on another model, which is generally difficult to control. As a first step in the BBOWP research, we chose a more controllable approach. This simple procedure allows us to easily control the precision–recall trade-off in identifier matching, while leaving room for improvement through additional alias lists. Combining these approaches could be a promising direction for further development.

D Details for Input and Output of BBOWP-Bench

D.1 Prompt Template for LLMs

We use the prompt template provided in Figure 3 as the input of BBOWP-Bench. For one-shot settings, we use the system prompt with the example based on Task 19, provided in Figure 4.

D.2 Task Description

As described in Section 4.2, we prepare three descriptions for each task with different difficulty levels (Easy, Intermediate, and Hard). We illustrate the example of the task descriptions of Task 3 (HPO of Ranger in YAHPO Gym) for the three difficulties in Figure 5.

D.3 Output Example

Figure 6 shows the example of an output for Task 4 with *Easy* difficulty and *Small* budget.

E Experimental Environment

Compute Resources. We executed the experiments on multiple computer resources as follows:

- Two computers running Ubuntu 22.04 with AMD EPYC7763 $\times$ 2 CPUs, 1024GB memory, and SSD 3.8TB.
- Two computers running Ubuntu 20.04 with AMD EPYC7763 $\times$ 1 CPU, 512GB memory, and SSD 3.8TB.
- Two computers running Ubuntu 22.04 with AMD EPYC7742 $\times$ 1 CPU, 512GB memory, and SSD 3.8TB.
- Three computers running Ubuntu 22.04 with Xeon E-2186G $\times$ 1 CPU, 64GB memory, and SSD 2TB.

Execution Time. For the *Small* budget ($N_{\max} = 100$), a single optimization run requires less than a minute for all tasks except Tasks 17 and 18 in MECHBench. For Tasks 17 and 18, some settings last for about 3 hours. For the *Large* budget ($N_{\max} = 10{,}000$), a single optimization run takes about 3 hours when the optimizer is not Bayesian optimization with Gaussian process (GP) or SMAC3. Some settings require more than 48 hours when using GP or SMAC3 as the optimizer, and we terminated optimization runs for such settings.

F Results of Algorithm Selection

Figure 7 shows the proportion of tasks for which each algorithm is selected by each LLM with the *Small* and *Large* budgets. The results demonstrate strong budget awareness across all models: sample-efficient surrogate methods (GP, SMAC3, and TPE) are favored under *Small* budget, whereas population-based evolution strategies (CMA-ES) dominate under *Large* budget.

Prompt Template

System Prompt:
Task
You are an expert in black-box optimization. The user will give you a task description of a black-box optimization problem. Your goal is to generate a JSON content that defines all of the following items based on the description:

1. search space
2. the best choice of an optimization algorithm, which must be selected from the predefined algorithms in the schema

Policy
To achieve the best result within the limited budget, please follow the policy below when generating the JSON content.

- For efficient optimization under the given budget, you can flexibly control the size of the search space by
    1. reducing the dimensionality of the search space (e.g., by fixing some parameters to be constant).
    2. adjusting the range of numerical parameters.
    3. reducing the number of choices for categorical parameters.
- You should choose the optimization algorithm that is most suitable for the defined search space and the given budget.

Output Format
Your output must be a JSON content that strictly follows the schema defined below.
{SCHEMA}
Additionally, the search space in the JSON content must be defined according to the following schema, where you can choose any combination of the variable types (int, float, bool, cat) and the constraints mentioned below:

- int: {"actual_name": "...", "type": "int", "low": 1, "high": 10, "step": 1, "log": false}
- float: {"actual_name": "...", "type": "float", "low": 1e-4, "high": 1.0, "log": true}
- bool: {"actual_name": "...", "type": "bool"}
- cat: {"actual_name": "...", "type": "cat", "choices": [{"actual_name": "a"}, {"actual_name": "b"}]}
- const_int: {"actual_name": "...", "type": "const_int", "value": 5}
- const_float: {"actual_name": "...", "type": "const_float", "value": 0.1}
- const_bool: {"actual_name": "...", "type": "const_bool", "value": true}
- const_cat: {"actual_name": "...", "type": "const_cat", "value": "gbtree"}

Make sure to include reasons for your search space design and algorithm choice in the JSON content, each of which should be written in the following aspects:

- search space design
    - the reason why each variable is included in the search space
    - the reason why each variable is optimized/fixed
    - the reason for the range/choices of each variable
- algorithm choice
    - the reason why the chosen algorithm is suitable for the defined search space and the given budget

Constraints

1. The name of each variable in the search space must be in snake_case.
2. The name of each choice for categorical variables must be in snake_case.

User Prompt:
Based on the task description given below, generate a JSON content according to the defined schema and system instructions.

Task Description
{TASK_DESCRIPTION}

Figure 3: Prompt template. {SCHEMA} defines the JSON schema for the output, specifying the available optimization algorithms, the required properties for each variable type, and the reasons behind the search space design and algorithm selection.

Prompt Template

System Prompt:
Task
...
Policy
...
Output Format
...
Constraints
...
Example
Task Description
{EXAMPLE_TASK_DESCRPITION}
Task Description
The objective is to maximize the number of timesteps an inverted pendulum remains balanced, simulated in MuJoCo via the Gymnasium interface (InvertedPendulum-v5). The control is performed by a linear policy that computes a horizontal force applied to the cart based on the current observation at each timestep. The episode terminates early if the absolute value of the pole angle exceeds 0.2 radians, and the maximum episode length is 1000 timesteps. The observation variables are: position (position of the cart along the linear track, in meters), angle (vertical angle of the pole relative to the upright position, in radians), velocity (linear velocity of the cart, in meters per second), and angular_velocity (angular velocity of the pole, in radians per second). The action variable force (horizontal force applied to the cart, in Newtons) is clipped to [-3.0, 3.0] by the simulator. The design variables consist of 5 continuous variables. Four of them are feedback gains that multiply each observation variable, and the remaining one is a bias term. The feedback gains are named as 'observation variable'_to_'action variable', and the bias term is named as bias_to_force. Each design variable ranges over [-1000, 1000]. Please note that while these specified ranges define the absolute implementational boundaries for each parameter, they serve primarily as a reference frame; you can flexibly control the actual search space depending on your optimization strategy. The evaluation budget is 100.

Output
{ "search_space": { "variables": [{ "actual_name": "position_to_force", "value": 0.0, "type": "const_float" }, { "actual_name": "angle_to_force", "low": -150.0, "high": 150.0, "log": false, "type": "float" }, { "actual_name": "velocity_to_force", "low": -6.0, "high": 6.0, "log": false, "type": "float" }, { "actual_name": "angular_velocity_to_force", "low": -30.0, "high": 30.0, "log": false, "type": "float" }, { "actual_name": "bias_to_force", "value": 0.0, "type": "const_float" }], "reason": "The search space is designed based on the scale of the action variable, which is bounded to [-3.0, 3.0] by the simulator, and has a margin to allow broad exploration. The reduction of the search space is needed due to the limited evaluation budget. The gain of angle, velocity, and angular velocity can be considered the three most important parameters for the control of the inverted pendulum, and the gain of position is fixed to 0.0 to reduce the search space. The bias term is fixed to 0.0 as this is expected to be less important." }, "algorithm": { "name": "TPE", "reason": "The search space involves three continuous variables. TPE is suitable for finding a reasonable solution within a limited budget and can handle continuous optimization problems." } }

Figure 4: System prompt for one-shot settings. The same parts as the prompt of zero-shot settings are omitted.

Task Descriptions of Task 3 (HPO of Ranger):

***Easy* difficulty:**
We need to optimize the hyperparameters of a random forest (Ranger) model to maximize AUC for a classification task. This optimization is formulated within an 8-dimensional mixed search space comprising continuous, integer, and categorical variables. It is important to consider the mechanics of ensemble learning and decision tree construction. Achieving robust predictive performance heavily relies on the discrete size of the tree ensemble, explicitly controlled by the variable 'num.trees' (typically evaluated within the bounds of 1 to 2000), alongside the continuous proportion of the training data sampled to build each individual tree, represented by the variable 'sample.fraction' (ranging from 0.1 to 1.0). Furthermore, the model's ability to capture complex patterns while maintaining diversity is governed by several factors: the strategy used to determine the subset of features evaluated at each split, defined by the variable 'mtry.power' (ranging from 0.0 to 1.0), the categorical rule chosen to measure split quality, corresponding to the variable 'splitrule' (with options being 'gini' or 'extratrees'), and the discrete number of random threshold evaluations per feature, controlled by the variable 'num.random.splits' (ranging from 1 to 100). To prevent the individual trees from growing too deep and overfitting, you must also carefully tune the discrete stopping criterion that defines the minimum allowed number of samples in a terminal leaf, which is managed by the variable 'min.node.size' (ranging from 1 to 100). Finally, the categorical approach taken to handle non-ordinal features, determined by the variable 'respect.unordered.factors' (accepting values like 'ignore', 'order', or 'partition'), combined with the preliminary strategy chosen to resolve missing data points, specified by the variable 'num.impute.selected.cpo' (with options 'impute.mean', 'impute.median', or 'impute.hist'), plays a critical role in the stability and ultimate performance of the resulting model. Please note that while these specified ranges and categorical options define the absolute implementational boundaries and available choices for each parameter, they serve primarily as a reference frame; you can flexibly control the actual search space depending on your optimization strategy.
The evaluation budget is {MAX_EVAL_VALUE}.

***Intermediate* difficulty:**
We need to optimize the hyperparameters of a random forest (Ranger) model to maximize AUC for a classification task. It is important to consider the mechanics of ensemble learning and decision tree construction. Achieving robust predictive performance heavily relies on the discrete size of the tree ensemble, alongside the continuous proportion of the training data sampled to build each individual tree. Furthermore, the model's ability to capture complex patterns while maintaining diversity is governed by several factors: the strategy used to determine the subset of features evaluated at each split, the categorical rule chosen to measure split quality, and the discrete number of random threshold evaluations per feature. To prevent the individual trees from growing too deep and overfitting, you must also carefully tune the discrete stopping criterion that defines the minimum allowed number of samples in a terminal leaf. Finally, the categorical approach taken to handle non-ordinal features, combined with the preliminary strategy chosen to resolve missing data points, plays a critical role in the stability and ultimate performance of the resulting model.
The evaluation budget is {MAX_EVAL_VALUE}.

***Hard* difficulty:**
We need to optimize the hyperparameters of Ranger (random forest) to maximize AUC.
The evaluation budget is {MAX_EVAL_VALUE}.

Figure 5: Task descriptions of Task 3 (HPO of Ranger in YAHPO) for the different difficulties, *Easy*, *Intermediate*, and *Hard*.

```
{
    "search_space": {
      "variables": [
        {"actual_name": "cp", "low": 0.0009118819655545162, "high": 1.0,
        "log": true, "type": "float"},
        {"actual_name": "maxdepth", "low": 1, "high": 30, "step": 1,
        "log": false, "type": "int"},
        {"actual_name": "minsplit", "low": 1, "high": 100, "step": 1,
        "log": false, "type": "int"},
        {"actual_name": "minbucket", "low": 1, "high": 100, "step": 1,
        "log": false, "type": "int"},
        {"actual_name": "num_impute_selected_cpo",
        "choices": [
          {"actual_name": "impute_mean"},
          {"actual_name": "impute_median"},
          {"actual_name": "impute_hist"}
        ],
        "type": "cat"
        }
      ],
      "reason": "The search space consists of 5 dimensions (one continuous, three integers, and one
      categorical variable). Given the generous evaluation budget of 10,000 trials, there is ample capacity
      to comprehensively explore the entire space without needing to freeze or narrow down any
      hyperparameters. Thus, all 5 variables are selected to be optimized across their originally
      prescribed ranges and choices, giving the optimization algorithm maximum flexibility to uncover
      complex interactions and effectively maximize the AUC. Variable names and categorical choices
      originally containing periods were converted to strictly follow snake_case format constraints."
    },
    "algorithm": {
      "name": "TPE",
      "reason": "Tree-structured Parzen Estimator (TPE) natively and robustly handles mixed search
      spaces encompassing continuous, integer, and categorical design variables. For this
      5-dimensional problem with a large evaluation budget of 10,000, TPE demonstrates superior
      scalability and computational efficiency compared to standard Gaussian Process-based
      techniques. It maintains an effective balance between exploration and exploitation across the
      thousands of trials, securing excellent convergence toward optimal hyperparameters while
      keeping algorithmic overhead manageable."
    }
}
```


Figure 6: Output for Task 4 with *Easy* difficulty and *Small* budget. Generated by Gemini 3 Pro Preview.

G Results of Optimization

G.1 Zero-Shot Settings

We conducted the experiments for all 23 tasks in BBOWP-Bench for *Small* budget ($N_{\max} = 100$). The results are provided in Table 5 for *Easy* difficulty, Table 6 for *Intermediate* difficulty, and Table 7 for *Hard* difficulty.

Additionally, we conducted the experiments for 21 tasks for *Large* budget ($N_{\max} = 10{,}000$). Tasks 17 and 18 in MECHBench are excluded because each evaluation takes a minute or more in those tasks. The results are provided in Table 8 for *Easy* difficulty, Table 9 for *Intermediate* difficulty, and Table 10 for *Hard* difficulty. We terminated the optimization runs, which took more than 48 hours, and omitted their results from each table, denoting such runs as "–".

Focusing on *Small* budget, there is no significant difference across the model size (Gemini 3 Flash vs. Gemini 3.1 Pro and GPT-5 mini vs. GPT-5.2) for all difficulties. We observe the same tendency for *Large* budget. In both budget settings, the best evaluation values on YAHPO tasks remain

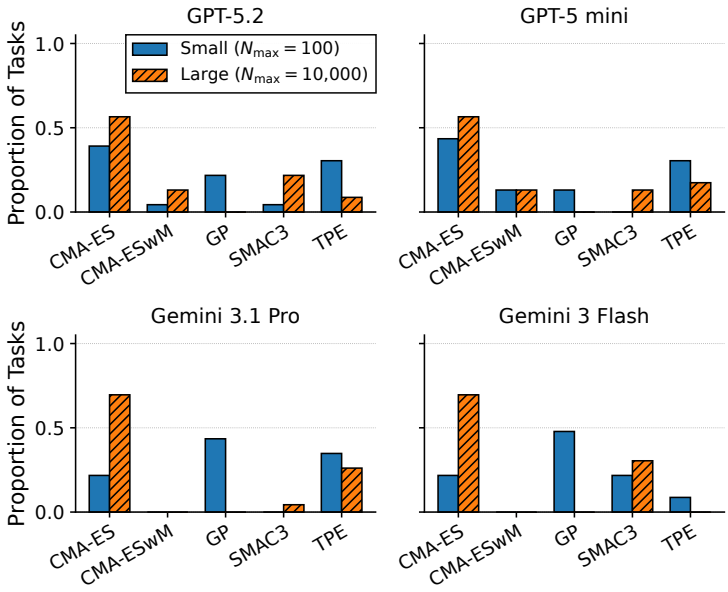


Figure 7: Distribution of optimization algorithms selected by the four evaluated LLMs across different evaluation budgets. Solid blue bars represent the *Small* budget ($N_{\max} = 100$), while striped orange bars represent the *Large* budget ($N_{\max} = 10{,}000$).

relatively stable, whereas they tend to decline as the settings become more challenging for Olympus, MECHBench, and MuJoCo.

G.2 One-Shot Settings

Similarly to zero-shot settings, we conducted the experiments for all 23 tasks in BBOWP-Bench for *Small* budget ($N_{\max} = 100$). We added Task 19's description and output into the system prompt as a one-shot example. The results are provided in Table 11 for *Easy* difficulty, Table 12 for *Intermediate* difficulty, and Table 13 for *Hard* difficulty.

Additionally, we conducted the experiments for 21 tasks for *Large* budget ($N_{\max} = 10{,}000$). Tasks 17 and 18 in MECHBench are excluded. Task 19 in MuJoCo is also omitted because it is included in the system prompt as the example. The results are provided in Table 14 for *Easy* difficulty, Table 15 for *Intermediate* difficulty, and Table 16 for *Hard* difficulty. We terminated the optimization runs that exceeded 48 hours and omitted their results from each table, denoting such runs as "–".

Compared with zero-shot prompting in *Easy* difficulty, one-shot prompting improves 8/16 settings for *Small* budget and 11/16 settings for *Large* budget, especially on Tasks 20-23, which share the same domain (MuJoCo) owing to the example of one-shot. Conversely, there are fewer settings where one-shot prompting improves the best evaluation value when the task is out of the domain of one-shot (YAHPO, Olympus, and MECHBench), both for *Small* and *Large* budgets.

Table 5: Comparison of optimization performance for *Easy* difficulty and *Small* budget.

Domain	Task	Gemini 3 Flash	Gemini 3.1 Pro	GPT-5 mini	GPT-5.2
YAHPO	Task 1 (↑)	**99.62**	99.57	**99.62**	99.31
	Task 2 (↑)	**0.96**	**0.96**	**0.96**	0.95
	Task 3 (↑)	0.92	0.87	0.94	**0.95**
	Task 4 (↑)	**0.92**	**0.92**	**0.92**	**0.92**
	Task 5 (↑)	**0.98**	0.94	0.97	0.94
	Task 6 (↑)	**1.00**	0.99	**1.00**	0.99
	Task 7 (↑)	**0.99**	0.96	**0.99**	**0.99**
Olympus	Task 8 (↓)	2.54	2.53	2.56	**2.52**
	Task 9 (↓)	**0.03**	**0.03**	0.04	0.07
	Task 10 (↓)	**0.02**	0.04	0.08	0.08
	Task 11 (↑)	0.95	0.95	0.95	**0.96**
	Task 12 (↓)	**0.00**	**0.00**	0.09	0.04
	Task 13 (↓)	**0.00**	**0.00**	**0.00**	**0.00**
	Task 14 (↓)	**0.19**	0.20	0.32	0.20
	Task 15 (↑)	**2695.47**	2364.52	2310.73	2246.71
	Task 16 (↑)	**99.81**	99.73	$-\infty$	98.44
MECHBench	Task 17 (↓)	**-11687.66**	**-11687.66**	-10758.91	-10758.91
	Task 18 (↓)	3.41	**3.39**	3.42	**3.39**
MuJoCo	Task 19 (↑)	**1000.00**	267.00	**1000.00**	187.70
	Task 20 (↑)	136.66	**150.58**	88.21	83.71
	Task 21 (↑)	270.05	884.00	**1009.84**	384.93
	Task 22 (↑)	-54.33	-60.92	**-12.25**	-14.68
	Task 23 (↑)	262.14	272.23	262.14	**324.97**

Table 6: Comparison of optimization performance for *Intermediate* difficulty and *Small* budget.

Domain	Task	Gemini 3 Flash	Gemini 3.1 Pro	GPT-5 mini	GPT-5.2
YAHPO	Task 1 (↑)	**99.52**	97.38	98.37	99.28
	Task 2 (↑)	0.95	**0.96**	0.93	0.92
	Task 3 (↑)	0.84	**0.88**	**0.88**	0.82
	Task 4 (↑)	0.90	**0.92**	0.91	0.91
	Task 5 (↑)	0.94	**0.95**	0.94	0.94
	Task 6 (↑)	**0.99**	**0.99**	**0.99**	**0.99**
	Task 7 (↑)	0.91	$-\infty$	0.91	**0.96**
Olympus	Task 8 (↓)	∞	**3.41**	∞	∞
	Task 9 (↓)	0.17	0.16	0.17	**0.05**
	Task 10 (↓)	**0.31**	**0.31**	0.32	**0.31**
	Task 11 (↑)	0.82	$-\infty$	$-\infty$	**0.92**
	Task 12 (↓)	0.33	0.33	0.33	**0.32**
	Task 13 (↓)	0.36	0.36	0.36	**0.35**
	Task 14 (↓)	∞	∞	**0.74**	0.94
	Task 15 (↑)	$-\infty$	$-\infty$	$-\infty$	**1283.76**
	Task 16 (↑)	**79.59**	$-\infty$	$-\infty$	69.42
MECHBench	Task 17 (↓)	-8503.76	**-8529.99**	800.31	800.31
	Task 18 (↓)	**3.61**	**3.61**	**3.61**	**3.61**
MuJoCo	Task 19 (↑)	367.10	**1000.00**	**1000.00**	**1000.00**
	Task 20 (↑)	99.20	**101.61**	92.94	75.02
	Task 21 (↑)	138.52	**1000.27**	138.52	138.52
	Task 22 (↑)	**-12.25**	**-12.25**	$-\infty$	$-\infty$
	Task 23 (↑)	**0.02**	**0.02**	**0.02**	**0.02**

Table 7: Comparison of optimization performance for *Hard* difficulty and *Small* budget.

Domain	Task	Gemini 3 Flash	Gemini 3.1 Pro	GPT-5 mini	GPT-5.2
YAHPO	Task 1 (↑)	98.01	**99.61**	$-\infty$	97.34
	Task 2 (↑)	0.93	**0.94**	**0.94**	0.92
	Task 3 (↑)	**0.88**	0.82	0.84	0.81
	Task 4 (↑)	0.91	0.91	**0.92**	**0.92**
	Task 5 (↑)	**0.95**	**0.95**	0.93	**0.95**
	Task 6 (↑)	**0.99**	**0.99**	**0.99**	**0.99**
	Task 7 (↑)	0.91	**0.92**	0.91	$-\infty$
Olympus	Task 8 (↓)	∞	∞	∞	∞
	Task 9 (↓)	**0.02**	0.03	0.05	0.03
	Task 10 (↓)	0.30	**0.05**	0.13	0.15
	Task 11 (↑)	0.94	**0.95**	**0.95**	0.80
	Task 12 (↓)	0.33	**0.32**	0.34	0.33
	Task 13 (↓)	0.36	**0.35**	0.36	**0.35**
	Task 14 (↓)	0.89	2.27	0.79	**0.78**
	Task 15 (↑)	$-\infty$	$-\infty$	$-\infty$	**1320.79**
	Task 16 (↑)	**55.25**	$-\infty$	$-\infty$	$-\infty$
MECHBench	Task 17 (↓)	**-8535.27**	-8528.84	∞	-8507.17
	Task 18 (↓)	**3.61**	**3.61**	**3.61**	**3.61**
MuJoCo	Task 19 (↑)	98.70	$-\infty$	24.00	**1000.00**
	Task 20 (↑)	**106.98**	$-\infty$	93.25	90.41
	Task 21 (↑)	**138.52**	**138.52**	**138.52**	**138.52**
	Task 22 (↑)	**-12.25**	**-12.25**	**-12.25**	**-12.25**
	Task 23 (↑)	**0.02**	-0.66	**0.02**	**0.02**

Table 8: Comparison of optimization performance for *Easy* difficulty and *Large* budget. The symbol "–" indicates that the run was terminated due to more than 48 hours of computational time.

Domain	Task	Gemini 3 Flash	Gemini 3.1 Pro	GPT-5 mini	GPT-5.2
YAHPO	Task 1 (↑)	–	**99.66**	**99.66**	99.57
	Task 2 (↑)	**0.97**	**0.97**	**0.97**	**0.97**
	Task 3 (↑)	0.96	**0.97**	0.96	0.96
	Task 4 (↑)	**0.92**	**0.92**	**0.92**	**0.92**
	Task 5 (↑)	–	**0.98**	–	–
	Task 6 (↑)	**1.00**	**1.00**	0.99	–
	Task 7 (↑)	–	–	**1.00**	–
Olympus	Task 8 (↓)	1.75	0.93	**0.05**	1.06
	Task 9 (↓)	**0.02**	0.03	**0.02**	**0.02**
	Task 10 (↓)	0.04	–	0.02	**0.01**
	Task 11 (↑)	**0.96**	**0.96**	**0.96**	**0.96**
	Task 12 (↓)	**0.00**	**0.00**	**0.00**	**0.00**
	Task 13 (↓)	**0.00**	**0.00**	**0.00**	**0.00**
	Task 14 (↓)	**0.20**	**0.20**	0.22	0.21
	Task 15 (↑)	2671.77	2621.12	**2759.50**	2732.99
	Task 16 (↑)	**101.42**	99.31	99.56	99.82
MuJoCo	Task 19 (↑)	**1000.00**	**1000.00**	**1000.00**	**1000.00**
	Task 20 (↑)	239.60	143.05	143.05	**315.08**
	Task 21 (↑)	1180.92	2589.99	1182.72	**2759.21**
	Task 22 (↑)	-71.12	**-5.40**	-66.62	-71.12
	Task 23 (↑)	355.17	**360.35**	0.02	355.17

Table 9: Comparison of optimization performance for *Intermediate* difficulty and *Large* budget. The symbol "–" indicates that the run was terminated due to more than 48 hours of computational time.

Domain	Task	Gemini 3 Flash	Gemini 3.1 Pro	GPT-5 mini	GPT-5.2
YAHPO	Task 1 (↑)	–	**99.66**	99.63	99.57
	Task 2 (↑)	–	**0.96**	**0.96**	–
	Task 3 (↑)	–	**0.91**	–	–
	Task 4 (↑)	–	**0.92**	**0.92**	**0.92**
	Task 5 (↑)	0.95	**0.96**	–	–
	Task 6 (↑)	–	**0.99**	–	–
Olympus	Task 8 (↓)	**1.13**	∞	∞	∞
	Task 9 (↓)	0.15	0.15	0.15	**0.09**
	Task 10 (↓)	0.30	0.30	0.30	**0.29**
	Task 11 (↑)	0.95	**0.96**	0.95	0.93
	Task 12 (↓)	**0.31**	**0.31**	–	**0.31**
	Task 13 (↓)	**0.34**	**0.34**	**0.34**	**0.34**
	Task 14 (↓)	**0.62**	∞	0.71	1.75
	Task 15 (↑)	$-\infty$	$-\infty$	1342.70	**1350.47**
	Task 16 (↑)	–	$-\infty$	**82.89**	–
MuJoCo	Task 19 (↑)	**1000.00**	**1000.00**	**1000.00**	**1000.00**
	Task 20 (↑)	100.72	100.72	119.98	**145.58**
	Task 21 (↑)	**138.52**	**138.52**	**138.52**	**138.52**
	Task 22 (↑)	**-12.25**	**-12.25**	**-12.25**	**-12.25**
	Task 23 (↑)	**0.02**	**0.02**	**0.02**	**0.02**

Table 10: Comparison of optimization performance for *Hard* difficulty and *Large* budget. The symbol "–" indicates that the run was terminated due to more than 48 hours of computational time.

Domain	Task	Gemini 3 Flash	Gemini 3.1 Pro	GPT-5 mini	GPT-5.2
YAHPO	Task 1 (↑)	**99.50**	$-\infty$	98.45	**99.50**
	Task 2 (↑)	–	**0.96**	–	–
	Task 3 (↑)	**0.91**	**0.91**	0.89	0.86
	Task 4 (↑)	–	**0.92**	–	–
	Task 5 (↑)	–	**0.96**	**0.96**	0.95
	Task 6 (↑)	–	**0.99**	**0.99**	$-\infty$
	Task 7 (↑)	–	–	–	**0.91**
Olympus	Task 8 (↓)	–	∞	∞	∞
	Task 9 (↓)	**0.02**	**0.02**	**0.02**	0.09
	Task 10 (↓)	0.11	**0.03**	0.19	0.10
	Task 11 (↑)	–	**0.96**	–	0.81
	Task 12 (↓)	0.32	0.32	0.31	**0.30**
	Task 13 (↓)	0.34	0.34	**0.33**	0.34
	Task 14 (↓)	–	**0.65**	–	–
	Task 15 (↑)	$-\infty$	$-\infty$	$-\infty$	**1380.15**
	Task 16 (↑)	–	$-\infty$	$-\infty$	–
MuJoCo	Task 19 (↑)	**1000.00**	**1000.00**	460.30	229.40
	Task 20 (↑)	119.96	$-\infty$	**323.52**	90.41
	Task 21 (↑)	–	**138.52**	**138.52**	**138.52**
	Task 22 (↑)	**-12.25**	**-12.25**	**-12.25**	**-12.25**
	Task 23 (↑)	**0.02**	**0.02**	**0.02**	**0.02**

Table 11: Comparison of optimization performance with one-shot prompting for *Easy* difficulty and *Small* budget.

Domain	Task	Gemini 3 Flash	Gemini 3.1 Pro	GPT-5 mini	GPT-5.2
YAHPO	Task 1 (↑)	97.01	96.12	**99.55**	98.11
	Task 2 (↑)	0.95	**0.96**	**0.96**	**0.96**
	Task 3 (↑)	0.88	0.86	0.91	**0.94**
	Task 4 (↑)	**0.92**	0.91	**0.92**	**0.92**
	Task 5 (↑)	0.95	0.95	**0.98**	0.95
	Task 6 (↑)	**0.99**	**0.99**	**0.99**	**0.99**
	Task 7 (↑)	0.98	0.97	0.99	**1.00**
Olympus	Task 8 (↓)	2.54	**2.30**	2.73	2.64
	Task 9 (↓)	**0.03**	**0.03**	**0.03**	0.06
	Task 10 (↓)	**0.04**	0.05	0.08	0.05
	Task 11 (↑)	**0.96**	**0.96**	0.95	0.95
	Task 12 (↓)	**0.00**	**0.00**	0.06	0.12
	Task 13 (↓)	**0.00**	**0.00**	0.09	0.01
	Task 14 (↓)	**0.19**	0.20	0.38	0.34
	Task 15 (↑)	2498.78	**2511.32**	2211.82	2295.25
	Task 16 (↑)	99.69	**100.86**	90.31	91.29
MECHBench	Task 17 (↓)	**-11687.66**	-11602.07	-11298.41	-10086.18
	Task 18 (↓)	3.41	3.41	4.68	**3.40**
MuJoCo	Task 20 (↑)	243.67	156.76	236.18	**376.58**
	Task 21 (↑)	999.21	836.11	**1021.98**	367.91
	Task 22 (↑)	-49.59	-90.06	-50.74	**-16.72**
	Task 23 (↑)	**357.92**	346.59	34.92	350.42

Table 12: Comparison of optimization performance with one-shot prompting for *Intermediate* difficulty and *Small* budget.

Domain	Task	Gemini 3 Flash	Gemini 3.1 Pro	GPT-5 mini	GPT-5.2
YAHPO	Task 1 (↑)	98.33	95.25	99.36	**99.45**
	Task 2 (↑)	0.95	**0.96**	0.94	0.92
	Task 3 (↑)	0.84	0.79	**0.89**	0.81
	Task 4 (↑)	0.90	**0.92**	**0.92**	0.91
	Task 5 (↑)	0.94	0.95	**0.96**	0.95
	Task 6 (↑)	**0.99**	**0.99**	**0.99**	**0.99**
	Task 7 (↑)	0.91	0.91	0.91	**0.92**
Olympus	Task 8 (↓)	9.06	**3.26**	∞	∞
	Task 9 (↓)	0.15	**0.12**	0.18	0.16
	Task 10 (↓)	**0.31**	**0.31**	**0.31**	**0.31**
	Task 11 (↑)	**0.95**	0.90	0.89	0.93
	Task 12 (↓)	0.34	0.34	**0.32**	0.33
	Task 13 (↓)	0.36	0.35	**0.34**	0.36
	Task 14 (↓)	∞	∞	**0.80**	0.91
	Task 15 (↑)	$-\infty$	$-\infty$	$-\infty$	**1274.15**
	Task 16 (↑)	$-\infty$	$-\infty$	**81.96**	60.14
MECHBench	Task 17 (↓)	**-8538.19**	-8528.84	∞	800.31
	Task 18 (↓)	**3.61**	**3.61**	**3.61**	**3.61**
MuJoCo	Task 20 (↑)	90.41	94.14	90.41	**176.75**
	Task 21 (↑)	998.72	138.52	138.52	**1055.56**
	Task 22 (↑)	**-12.25**	**-12.25**	**-12.25**	$-\infty$
	Task 23 (↑)	**0.02**	**0.02**	**0.02**	**0.02**

Table 13: Comparison of optimization performance with one-shot prompting for *Hard* difficulty and *Small* budget.

Domain	Task	Gemini 3 Flash	Gemini 3.1 Pro	GPT-5 mini	GPT-5.2
YAHPO	Task 1 (↑)	**97.78**	96.52	$-\infty$	$-\infty$
	Task 2 (↑)	**0.94**	0.93	0.93	0.92
	Task 3 (↑)	**0.86**	0.76	0.84	0.83
	Task 4 (↑)	**0.92**	**0.92**	**0.92**	0.91
	Task 5 (↑)	**0.95**	**0.95**	0.94	**0.95**
	Task 6 (↑)	**0.99**	$-\infty$	**0.99**	**0.99**
	Task 7 (↑)	**0.91**	**0.91**	**0.91**	**0.91**
Olympus	Task 8 (↓)	∞	∞	∞	∞
	Task 9 (↓)	**0.03**	0.05	0.04	0.04
	Task 10 (↓)	0.31	**0.12**	0.13	0.31
	Task 11 (↑)	**0.95**	**0.95**	**0.95**	0.80
	Task 12 (↓)	**0.33**	0.34	**0.33**	**0.33**
	Task 13 (↓)	**0.35**	0.36	0.36	0.36
	Task 14 (↓)	0.83	0.84	**0.79**	1.23
	Task 15 (↑)	$-\infty$	$-\infty$	**1316.15**	1257.63
	Task 16 (↑)	$-\infty$	$-\infty$	$-\infty$	$-\infty$
MECHBench	Task 17 (↓)	**-8503.76**	-8482.05	∞	800.31
	Task 18 (↓)	**3.61**	**3.61**	**3.61**	**3.61**
MuJoCo	Task 20 (↑)	92.00	98.08	**189.01**	96.43
	Task 21 (↑)	**138.52**	**138.52**	**138.52**	**138.52**
	Task 22 (↑)	**-12.25**	**-12.25**	**-12.25**	$-\infty$
	Task 23 (↑)	**0.02**	**0.02**	**0.02**	**0.02**

Table 14: Comparison of optimization performance with one-shot prompting for *Easy* difficulty and *Large* budget. The symbol "–" indicates that the run was terminated due to more than 48 hours of computational time.

Domain	Task	Gemini 3 Flash	Gemini 3.1 Pro	GPT-5 mini	GPT-5.2
YAHPO	Task 1 (↑)	–	**99.66**	$-\infty$	99.57
	Task 2 (↑)	**0.96**	**0.96**	**0.96**	**0.96**
	Task 3 (↑)	**0.96**	0.91	**0.96**	**0.96**
	Task 4 (↑)	**0.92**	**0.92**	**0.92**	**0.92**
	Task 5 (↑)	0.96	**0.98**	**0.98**	0.96
	Task 6 (↑)	–	0.99	**1.00**	0.99
	Task 7 (↑)	–	**1.00**	–	**1.00**
Olympus	Task 8 (↓)	**0.30**	0.83	2.20	1.97
	Task 9 (↓)	**0.02**	**0.02**	**0.02**	**0.02**
	Task 10 (↓)	**0.00**	0.01	0.03	0.03
	Task 11 (↑)	**0.96**	**0.96**	**0.96**	**0.96**
	Task 12 (↓)	**0.00**	**0.00**	–	**0.00**
	Task 13 (↓)	**0.00**	**0.00**	**0.00**	**0.00**
	Task 14 (↓)	0.22	0.23	**0.21**	0.27
	Task 15 (↑)	2532.92	2692.48	**2704.45**	2568.60
	Task 16 (↑)	**100.00**	97.88	97.58	99.92
MuJoCo	Task 20 (↑)	443.79	456.84	9359.43	**9359.86**
	Task 21 (↑)	1029.72	**2589.99**	1199.98	1347.54
	Task 22 (↑)	-5.84	-28.56	-6.00	**-5.04**
	Task 23 (↑)	359.10	360.35	108.94	**361.33**

Table 15: Comparison of optimization performance with one-shot prompting for *Intermediate* difficulty and *Large* budget. The symbol "–" indicates that the run was terminated due to more than 48 hours of computational time.

Domain	Task	Gemini 3 Flash	Gemini 3.1 Pro	GPT-5 mini	GPT-5.2
YAHPO	Task 1 (↑)	98.53	**99.57**	99.50	**99.57**
	Task 2 (↑)	–	**0.96**	–	–
	Task 3 (↑)	–	**0.91**	–	0.89
	Task 4 (↑)	–	**0.92**	**0.92**	–
	Task 5 (↑)	–	–	–	**0.95**
	Task 6 (↑)	**0.99**	**0.99**	–	**0.99**
	Task 7 (↑)	–	–	–	–
Olympus	Task 8 (↓)	0.22	**0.16**	∞	3.00
	Task 9 (↓)	0.15	0.15	0.15	**0.03**
	Task 10 (↓)	**0.29**	0.30	0.30	0.30
	Task 11 (↑)	**0.96**	**0.96**	**0.96**	0.93
	Task 12 (↓)	**0.30**	0.31	**0.30**	0.31
	Task 13 (↓)	**0.34**	**0.34**	**0.34**	**0.34**
	Task 14 (↓)	**0.63**	∞	0.68	∞
	Task 15 (↑)	$-\infty$	$-\infty$	$-\infty$	–
	Task 16 (↑)	81.46	$-\infty$	**83.81**	83.39
MuJoCo	Task 20 (↑)	112.44	123.07	**363.04**	123.07
	Task 21 (↑)	**1039.23**	138.52	138.52	138.52
	Task 22 (↑)	**-12.25**	**-12.25**	**-12.25**	$-\infty$
	Task 23 (↑)	**0.02**	**0.02**	**0.02**	**0.02**

Table 16: Comparison of optimization performance with one-shot prompting for *Hard* difficulty and *Large* budget. The symbol "–" indicates that the run was terminated due to more than 48 hours of computational time.

Domain	Task	Gemini 3 Flash	Gemini 3.1 Pro	GPT-5 mini	GPT-5.2
YAHPO	Task 1 (↑)	–	**99.57**	$-\infty$	$-\infty$
	Task 2 (↑)	–	0.93	–	**0.96**
	Task 3 (↑)	0.86	**0.91**	0.87	–
	Task 4 (↑)	–	**0.92**	**0.92**	–
	Task 5 (↑)	–	**0.95**	**0.95**	**0.95**
	Task 6 (↑)	–	**0.99**	–	**0.99**
	Task 7 (↑)	–	–	0.91	**0.96**
Olympus	Task 8 (↓)	–	∞	–	∞
	Task 9 (↓)	0.02	0.02	**0.01**	0.02
	Task 10 (↓)	0.30	**0.02**	0.11	0.12
	Task 11 (↑)	–	**0.96**	–	–
	Task 12 (↓)	0.31	0.32	0.31	**0.30**
	Task 13 (↓)	**0.33**	**0.33**	0.34	**0.33**
	Task 14 (↓)	–	**0.66**	–	–
	Task 15 (↑)	–	–	$-\infty$	**1390.82**
	Task 16 (↑)	–	–	–	–
MuJoCo	Task 20 (↑)	100.72	88.20	**323.52**	260.96
	Task 21 (↑)	138.52	**1104.08**	138.52	138.52
	Task 22 (↑)	**-12.25**	**-12.25**	**-12.25**	$-\infty$
	Task 23 (↑)	**0.02**	**0.02**	-0.26	**0.02**

H Search Spaces Designed by LLMs

Tables 17 to 21 present representative examples of the search space designed by the LLMs for specified tasks and settings: Table 17 corresponds to Task 4 (HPO of SVM in YAHPO) in *Hard* difficulty under *Small* budget; Table 18 corresponds to Task 14 (S_NAr reaction in Olympus) under *Hard* difficulty under *Small* budget; Table 19 corresponds to Task 17 (Star-shaped crash box, MECHBench) in *Hard* difficulty under *Small* budget; Tables 20 and 21 correspond to Task 20 (Inverted Double Pendulum, MuJoCo) in *Hard* and *Easy* difficulty under *Small* budget, respectively.

For Task 4 (HPO of SVM in YAHPO), the best objective values of all LLMs and the human baseline are comparable since all LLMs correctly inferred several important variables in their search space, such as `cost`, `kernel`, and `gamma`. However, only GPT-5 mini yielded a narrow range restricted to small values of [1e-4, 1.0] for `cost` and *gamma*, which could result in a slightly lower best objective value than those of the other LLMs.

For Task 14 (S_NAr reaction in Olympus), the best objective values achieved by the LLM-designed search spaces and selected algorithms remained below that of the human baseline. This gap is attributable to the suboptimal search space designs proposed by the LLMs: although most models correctly identified one or both of `residence_time` and `temperature` as design variables, the specified value ranges were poorly calibrated. Excessively long residence times promote side reactions, thereby increasing impurity, while excessively low temperatures suppress the reaction rate and reduce substrate conversion, both of which are detrimental to impurity minimization.

The best objective values achieved by the LLM-designed search spaces and selected algorithms also fell short of the human baseline in Task 17 (Star-shaped crash box, MECHBench). In the LLM-designed search spaces, only the variable `thickness` was mapped to the canonical variables. The best objective value of GPT-5 mini was $-\infty$ because its value range of [5e-4, 2e-2] was outside the canonical variable range, and those of the other LLMs did not reach the human baseline. The LLMs designed their search spaces based on a polar coordinate system, while a rectangular coordinate system was expected in the canonical design space. This variation in coordinate system conventions for designing search space presents a significant challenge that lies beyond the scope of the current framework. This topic is one of the directions of future work.

For Task 20 (Inverted Double Pendulum, MuJoCo), some LLMs tended to design their search spaces with some or all variables asymmetric about zero in both *Hard* and *Easy* difficulty. This asymmetric design potentially demonstrates that LLMs leverage their knowledge about control engineering for designing search spaces, and contributed to superior optimization results of Gemini 3 Flash and Gemini 3.1 Pro over that of the human baseline in *Easy* difficulty.

Based on the analyses above, although inferring correct variables and value ranges is highly challenging for LLMs in general, the best objective values in several tasks are comparable to those of the human baseline. These results stem from the design of effective search spaces that leverage the domain knowledge LLMs possess.

Table 17: Comparison of search space designed by human and LLMs, and the best achieved objective value for Task 4 (HPO of SVM in YAHPO) in *Hard* difficulty and *Small* budget. The symbol "–" indicates the variable was not included in the search space provided by the model.

Variables	Canonical Domain	Human Baseline	Gemini 3 Flash	Gemini 3.1 Pro	GPT-5 mini	GPT-5.2
cost	[4.54e-5, 22026.47]	[4.54e-5, 22026.47], log	[1e-3, 1e3], log	[1e-3, 1e3], log	[1e-4, 1.0], log	[1e-3, 1e3], log
kernel	{linear, radial, polynomial}	{linear, radial, polynomial}	{linear, radial, polynomial}	{linear, radial}	{linear, radial, polynomial}	{linear, radial}
tolerance	[4.54e-5, 2.0]	[4.54e-5, 2.0], log	–	–	–	[1e-5, 1e-2], log
degree	[2, 5], step=1	[2, 5], step=1	–	–	[2, 5], step=2	–
gamma	[4.54e-5, 22026.47], log	[4.54e-5, 22026.47], log	[1e-4, 10.0], log	[1e-4, 10.0], log	[1e-4, 1.0], log	[1e-4, 10.0], log
num.impute.selected.cpo	{mean, hist, median}	hist	–	–	–	–
Additional variables						
probability	–	–	True	True	–	False
coef0	–	–	–	–	0.0	–
class_weight	–	–	–	–	–	{none, balanced}
shrinking	–	–	–	–	–	{True, False}
max_iter	–	–	–	–	–	-1
Best obj. (↑)	–	0.9773	0.9493	0.9516	0.9281	0.9503

Table 18: Comparison of search space designed by human and LLMs, and the best achieved objective value for Task 14 (S_NAr reaction in Olympus) in *Hard* difficulty and *Small* budget. The symbol "–" indicates the variable was not included in the search space provided by the model.

Variables	Canonical Domain	Human Baseline	Gemini 3 Flash	Gemini 3.1 Pro	GPT-5 mini	GPT-5.2
residence_time	[0.5, 2.0]	[0.5, 2.0]	[0.0, 1440]	[1.0, 24.0]	–	–
ratio	[1.0, 5.0]	[1.0, 5.0]	–	–	–	–
concentration	[0.1, 0.5]	[0.1, 0.5]	[0.01, 2.0], log	–	[0.01, 1.0], log	[0.05, 1.0], log
temperature	[60.0, 140.0]	[60.0, 140.0]	[0.0, 250.0]	[20.0, 140.0]	[20, 150], step=1	[20.0, 140.0]
Additional variables						
solvent	–	–	{dmf, dmso, nmp, thf, acetonitrile}	{dmf, dmso, nmp}	{ dimethylformamide, dimethylsulfoxide, acetonitrile}	{dmf, dmso, acetonitrile, nmp}
base_equivalents	–	–	[0.5, 5.0]	–	–	[0.5, 3.0]
base_type	–	–	{k2co3, cs2co3, tea, naoh, koh}	{k2co3, cs2co3, dipea}	{potassium carbonate, cesium carbonate, triethylamine}	{k2co3, cs2co3, dipea}
nucleophile equivalents	–	–	–	[1.0, 3.0]	[1.0, 3.0]	[0.8, 2.0]
reaction_time_min	–	–	–	–	[15, 480], step=5	–
reaction_time_h	–	–	–	–	–	[0.5, 24.0]
Best obj. (↓)	–	0.1990	0.8937	2.2738	0.7923	0.7771

Table 19: Comparison of search space designed by human and LLMs, and the best achieved objective value for Task 17 (Star-shaped crash box, MECHBench) in *Hard* difficulty and *Small* budget. The symbol "–" indicates the variable was not included in the search space provided by the model.

Variables	Canonical Domain	Human Baseline	Gemini 3 Flash	Gemini 3.1 Pro	GPT-5 mini	GPT-5.2
width	[60, 120]	[60, 120]	–	–	–	–
depth	[60, 120]	[60, 120]	–	–	–	–
notch_width	[0, 30]	15	–	–	–	–
notch_depth	[0, 30]	15	–	–	–	–
thickness	[0.7, 3.0]	[1.1, 2.4]	[1.0, 3.0]	[1.0, 5.0]	[5e-4, 2e-2]	[0.8, 4.0], log
Additional variables						
inner_radius	–	–	[20.0, 40.0]	[20.0, 60.0]	[1e-2, 0.2]	[0.35, 0.85]
outer_radius	–	–	[41.0, 60.0]	[65.0, 120.0]	[5e-2, 0.3]	[25.0, 75.0]
corner_radius	–	–	[2.0, 10.0]	–	–	[0.5, 8.0]
number_of_corners	–	–	4	–	–	–
length	–	–	–	200.0	–	–
num_points	–	–	–	–	[3, 12], step=1	[4, 10], step=1
crash_box_height	–	–	–	–	–	[80.0, 220.0]
inner_radius_ratio	–	–	–	–	–	[0.35, 0.85]
taper_angle_deg	–	–	–	–	–	[0.0, 6.0]
Best obj. (↓)	–	-11740.92	-8535.27	-8528.84	∞	-8507.17

Table 20: Comparison of search space designed by human and LLMs, and the best achieved objective value for Task 20 (Inverted Double Pendulum, MuJoCo) in *Hard* difficulty and *Small* budget. The symbol "−" indicates the variable was not included in the search space provided by the model.

Variables	Canonical Domain	Human Baseline	Gemini 3 Flash	Gemini 3.1 Pro	GPT-5 mini	GPT-5.2
position_to_force	[-1e3, 1e3]	0	[-50, 0]	−	[0, 10]	−
sin_pole1_to_force	[-1e3, 1e3]	[-10, 10]	−	−	−	−
sin_pole2_to_force	[-1e3, 1e3]	[-10, 10]	−	−	−	−
cos_pole1_to_force	[-1e3, 1e3]	0	−	−	−	−
cos_pole2_to_force	[-1e3, 1e3]	0	−	−	−	−
velocity_to_force	[-1e3, 1e3]	0	[-20, 0]	−	−	−
angular_velocity pole1_to_force	[-1e3, 1e3]	[-50, 50]	−	−	−	−
angular_velocity pole2_to_force	[-1e3, 1e3]	[-50, 50]	−	−	−	−
constraint_force _to_force	[-1e3, 1e3]	0	−	−	−	−
bias_to_force	[-1e3, 1e3]	0	−	−	[-10, 10]	[-2, 2]
Additional variables						
k_pole1_ang	−	−	[10, 300]	−	−	−
k_pole1_vel	−	−	[2, 100]	−	−	−
k_pole2_ang	−	−	[10, 300]	−	−	−
k_pole2_vel	−	−	[2, 100]	−	−	−
w0	−	−	−	[-10, 10]	−	−
w1	−	−	−	[-10, 10]	−	−
w2	−	−	−	[-10, 10]	−	−
w3	−	−	−	[-10, 10]	−	−
w4	−	−	−	[-10, 10]	−	−
w5	−	−	−	[-10, 10]	−	−
w6	−	−	−	[-10, 10]	−	−
w7	−	−	−	[-10, 10]	−	−
w8	−	−	−	[-10, 10]	−	−
w9	−	−	−	[-10, 10]	−	−
w10	−	−	−	[-10, 10]	−	−
kp_pole1	−	−	−	−	[0, 50]	−
kp_pole2	−	−	−	−	[0, 50]	−
kd_angles	−	−	−	−	[0, 20]	−
w_cart_position	−	−	−	−	−	[-5, 5]
w_cart_velocity	−	−	−	−	−	[-5, 5]
w_pole_1_angle	−	−	−	−	−	[-10, 10]
w_pole_1 angular_velocity	−	−	−	−	−	[-5, 5]
w_pole_2_angle	−	−	−	−	−	[-10, 10]
w_pole_2_angular velocity	−	−	−	−	−	[-5, 5]
Best obj. (↑)	−	135.08	106.98	$-\infty$	93.25	90.41

Table 21: Comparison of search space designed by human and LLMs, and the best achieved objective value for Task 20 (Inverted Double Pendulum, MuJoCo, maximization) in *Easy* difficulty and *Small* budget. The symbol "−" indicates the variable was not included in the search space provided by the model.

Variables	Canonical Domain	Human Baseline	Gemini 3 Flash	Gemini 3.1 Pro	GPT-5 mini	GPT-5.2
position_to_force	[-1e3, 1e3]	0	[-5e2, 5e2]	[-50, 50]	[-2e2, 2e2]	[-1e2, 1e2]
sin_pole1_to_force	[-1e3, 1e3]	[-10, 10]	[-1e3, 1e3]	[-50, 50]	[-2e2, 2e2]	[-1e2, 1e2]
sin_pole2_to_force	[-1e3, 1e3]	[-10, 10]	[-1e3, 1e3]	[-50, 50]	[-2e2, 2e2]	[-1e2, 1e2]
cos_pole1_to_force	[-1e3, 1e3]	0	[-5e2, 5e2]	0	[-2e2, 2e2]	[-1e2, 1e2]
cos_pole2_to_force	[-1e3, 1e3]	0	[-5e2, 5e2]	0	[-2e2, 2e2]	[-1e2, 1e2]
velocity_to_force	[-1e3, 1e3]	0	[-2e2, 2e2]	[-50, 50]	[-2e2, 2e2]	[-1e2, 1e2]
angular_velocity pole1_to_force	[-1e3, 1e3]	[-50, 50]	[-2e2, 2e2]	[-50, 50]	[-2e2, 2e2]	[-1e2, 1e2]
angular_velocity pole2_to_force	[-1e3, 1e3]	[-50, 50]	[-2e2, 2e2]	[-50, 50]	[-2e2, 2e2]	[-1e2, 1e2]
constraint_force _to_force	[-1e3, 1e3]	0	[-10, 10]	0	[-2e2, 2e2]	[-1e2, 1e2]
bias_to_force	[-1e3, 1e3]	0	[-10, 10]	0	[-2e2, 2e2]	[-50, 50]
Best obj.	−	135.08	136.66	150.58	88.21	83.71

I Results of FRS Mertic

We calculate the FRS metrics for Tasks 3, 14, and 20 under *Small* budget. The complete results are provided in Tables 22 and 23 with zero-shot prompting and one-shot prompting, respectively. Focusing on the differences in FRS values between Tables 22 and 23, the one-shot prompting improves the average FRS of Task 20 for the models other than Gemini 3 Flash, since we provide the example in the prompt using Task 19, which shares its domain with Task 20 (MuJoCo). Conversely, one-shot prompting sometimes deteriorates the FRS values for Tasks 3 and 14, whose domain differs from that of the example.

Table 22: Comparison of FRS between different models in *Small* budget and zero-shot prompting setting.

Difficulty	Model	Task 3	Task 14	Task 20	Avg. (Task)
Easy	Gemini 3 Flash	0.999	1.000	0.896	0.965
	Gemini 3.1 Pro	0.887	1.000	0.922	0.936
	GPT-5 mini	1.000	0.994	0.570	0.855
	GPT-5.2	1.000	1.000	0.530	0.843
Intermediate	Gemini 3 Flash	0.615	0.000	0.687	0.434
	Gemini 3.1 Pro	0.958	0.000	0.714	0.557
	GPT-5 mini	0.952	0.617	0.620	0.730
	GPT-5.2	0.359	0.425	0.463	0.416
Hard	Gemini 3 Flash	0.953	0.469	0.763	0.728
	Gemini 3.1 Pro	0.393	0.005	0.000	0.133
	GPT-5 mini	0.577	0.561	0.624	0.588
	GPT-5.2	0.341	0.576	0.594	0.503
Avg. (Difficulty)	Gemini 3 Flash	0.856	0.490	0.782	0.709
	Gemini 3.1 Pro	0.746	0.335	0.545	0.542
	GPT-5 mini	0.843	0.724	0.605	0.724
	GPT-5.2	0.567	0.667	0.529	0.588

Table 23: Comparison of FRS between different models in *Small* budget and one-shot prompting setting.

Difficulty	Model	Task 3	Task 14	Task 20	Avg. (Task)
Easy	Gemini 3 Flash	0.944	1.000	0.980	0.975
	Gemini 3.1 Pro	0.795	1.000	0.931	0.909
	GPT-5 mini	0.998	0.971	0.978	0.983
	GPT-5.2	1.000	0.988	0.996	0.995
Intermediate	Gemini 3 Flash	0.604	0.000	0.594	0.399
	Gemini 3.1 Pro	0.148	0.000	0.633	0.260
	GPT-5 mini	0.985	0.556	0.594	0.712
	GPT-5.2	0.304	0.454	0.951	0.570
Hard	Gemini 3 Flash	0.794	0.523	0.609	0.642
	Gemini 3.1 Pro	0.058	0.517	0.676	0.417
	GPT-5 mini	0.559	0.561	0.959	0.693
	GPT-5.2	0.492	0.223	0.656	0.457
Avg. (Difficulty)	Gemini 3 Flash	0.781	0.508	0.728	0.672
	Gemini 3.1 Pro	0.334	0.506	0.747	0.529
	GPT-5 mini	0.847	0.696	0.844	0.796
	GPT-5.2	0.598	0.555	0.868	0.674

J Results of Search Space Quality

Figures 8 and 9 visualize how the objective function values distribute when sampling 100,000 points from the canonical domain of the task and search spaces designed by the LLMs with zero-shot and one-shot prompting, respectively.

In both zero-shot and one-shot settings, for Task 3 in *Easy* difficulty, the median values of the samples from the search spaces designed by LLMs are better than those from the canonical domains. Conversely, in *Intermediate* and *Hard* difficulty, where more implicit descriptions are given, the median values for the search spaces by LLMs deteriorate even with one-shot prompting.

Comparing zero-shot and one-shot settings, one-shot prompting improves the median values slightly in *Easy* and *Intermediate* difficulties for Task 20, which shares its domain with the one-shot example. However, the effectiveness of one-shot prompting is limited as seen in *Hard* difficulty for Task 20 and the difficulties for Tasks 3 and 14.

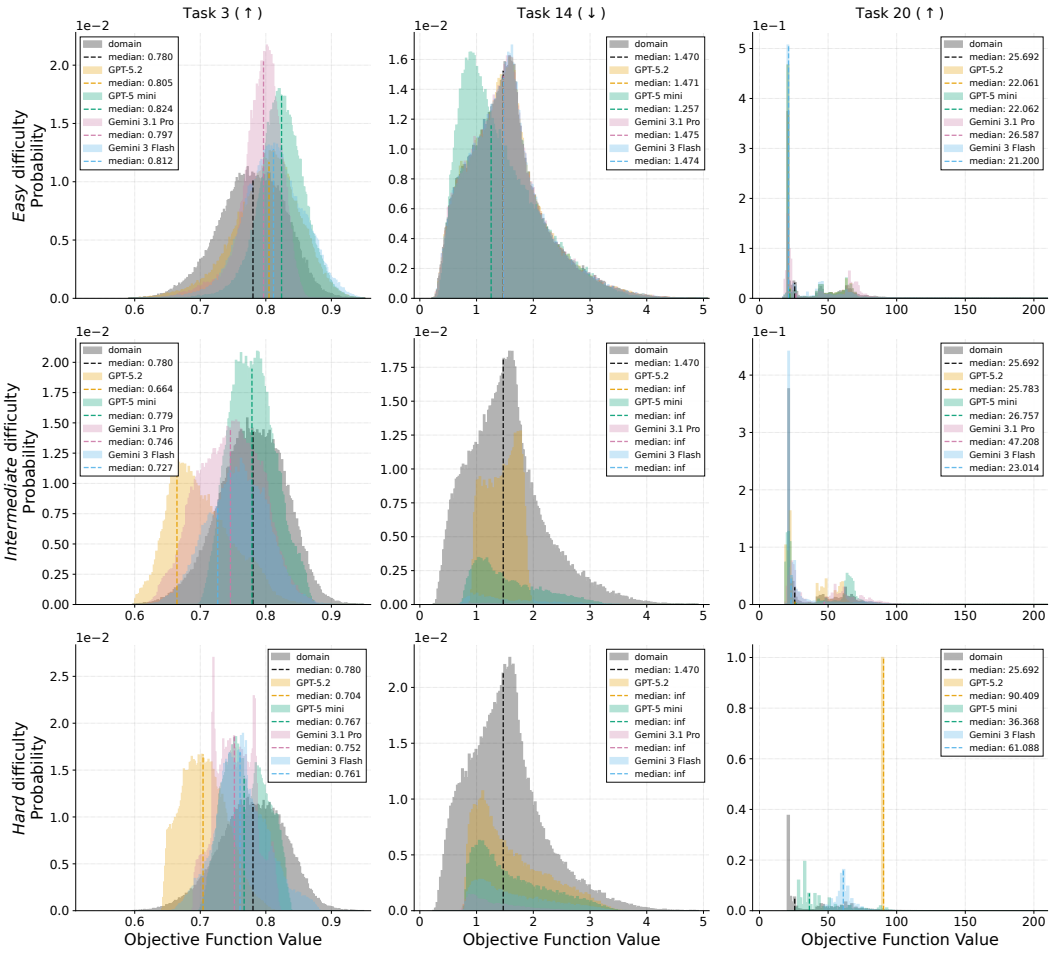


Figure 8: EPDF of objective function values obtained by uniformly sampling 100,000 points from the canonical domain (gray) versus the search spaces generated by four LLMs. This result is the case of zero-shot prompting. The worst objective function value is omitted in the figure.

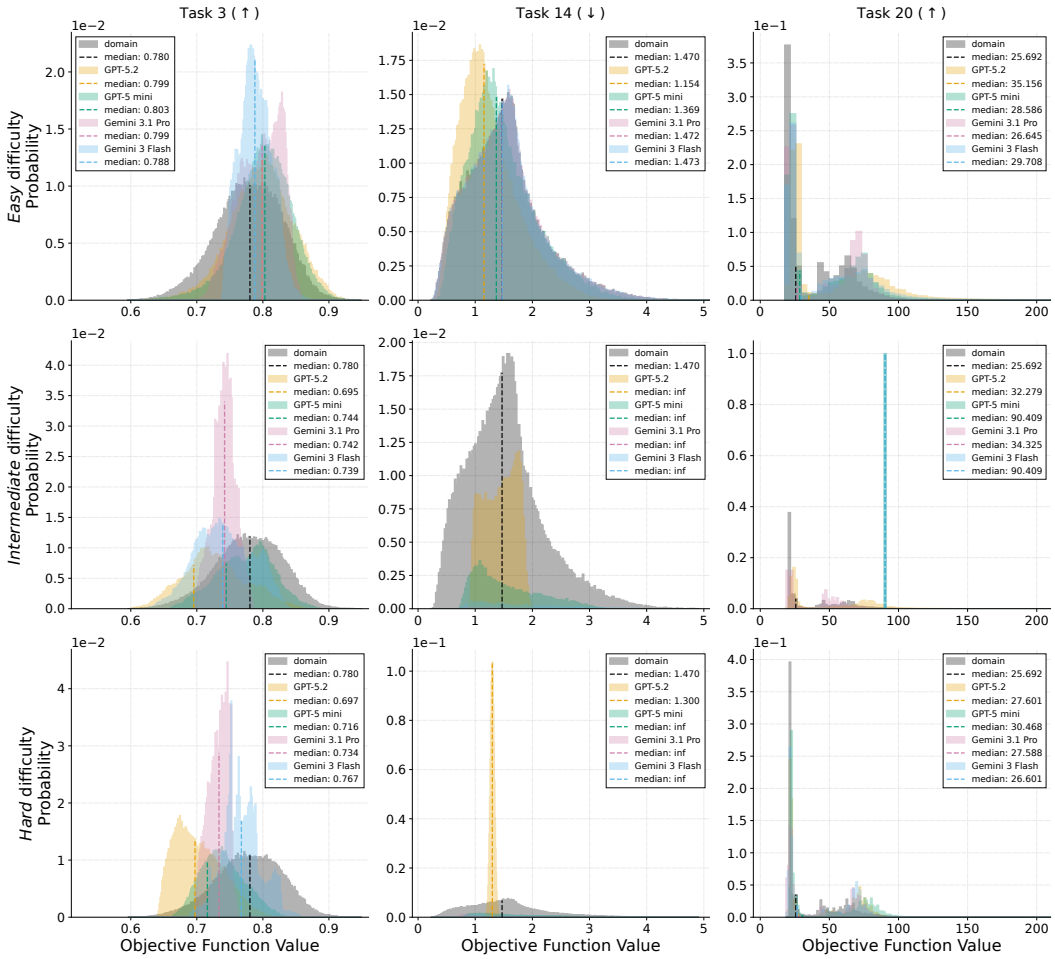


Figure 9: EPDF of objective function values obtained by uniformly sampling 100,000 points from the canonical domain (gray) versus the search spaces generated by four LLMs. This result is the case of one-shot prompting. The worst objective function value is omitted in the figure.

K Potential Societal Impact

We believe that our proposed benchmark can support research on automated formulation for black-box optimization, potentially reducing the expertise required to apply optimization methods. A possible negative societal impact is that more automated optimization could be misused in harmful applications if combined with inappropriate objectives or unsafe evaluation environments. However, this paper introduces a benchmark and does not release high-risk models, scraped datasets, or deployed optimization systems.